



M362 Unit 8

UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



The web tier

Unit 8

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1595 8

1.1

CONTENTS

1	Introduction	6
1.1	The aims of this unit	6
2	Servlets	7
2.1	The need for servlets	7
2.2	How servlets run	10
2.3	How servlets are programmed	11
2.4	Processing GET and POST requests	16
2.5	Dealing with concurrent requests	17
2.6	Using sessions with servlets	19
2.7	Cookies and sessions	20
3	JavaServer Pages	23
3.1	Motivation for JSP	23
3.2	JSP syntax	23
3.3	Examples of JSP	25
3.4	How JSP works	29
4	JavaBeans	31
4.1	What a JavaBean is	31
4.2	JavaBeans and JSP	32
4.3	JavaBeans example	33
4.4	JavaBeans and servlets	35
5	More on JavaServer Pages	36
5.1	JSP tag libraries	36
5.2	JSP expression language	38
6	Structuring the web tier	43
6.1	Model 1 and Model 2 approaches	43
6.2	Communication between components in an MVC architecture	44
6.3	MVC frameworks	48

Continued over page

7	Communication with the business tier	50
8	The Music Store scenario	53
9	Summary	55
	Glossary	57
	References	61
	Index	62



M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

We have seen that Java EE systems are constructed in three main tiers: client tier, middle tier and data tier. The middle tier is normally divided into two subcomponents, the web tier and the business tier. This unit is about the **web tier**, the part of the middle tier that communicates with the client tier, typically by receiving and responding to HTTP requests.

The code associated with the web tier runs on the server machine, not on the client machine. It sits between the **business tier**, also on the server, and the client tier software which runs directly on the client machine, normally in the form of a web browser. As we saw in the previous unit, the business tier contains the Enterprise JavaBeans (EJB) components that deal with the business logic of the system.

In some Java EE systems, the web tier may be omitted, especially if the client tier software is a specially designed application providing a suitable user interface for the system – sometimes called a **fat client**. This approach is considered in the next unit, which discusses the client tier.

The primary function of the web tier is to receive requests from the client software and respond by constructing web pages dynamically. To do this it makes use of **servlets**, Java programs that run on the server and that also interact with the business tier. The web tier also typically makes use of **JavaServer Pages (JSP)**. Both servlets and JSP can be used to generate web pages dynamically, producing web pages with a mixture of static content together with items that are updated as required.

Components in the web tier, such as servlets and **JSP pages**, communicate with the client tier using HTTP, the standard protocol for the Web. The link from the web tier to the EJB components in the business tier is by means of the business interface of EJB session beans, as described in *Unit 7*.

HTTP stands for Hypertext Transfer Protocol and defines rules for communication on the Web.

1.1 The aims of this unit

In this unit you will study:

- ▶ the function of the web tier;
- ▶ what servlets are and how they work;
- ▶ how JSP technology can be used to generate web pages dynamically;
- ▶ when to use servlets, JSP, or a combination of both;
- ▶ communication between the web tier and adjacent tiers;
- ▶ concurrency and distribution issues for the web tier.

This unit involves reading, and practical activities related to programming the web tier. In particular, you will continue with the case study of The Music Store.

2

Servlets

Most of the operation of the Java EE web tier depends on servlets in a variety of ways. In this section we consider why servlets are useful in web-based systems and how they work.

2.1 The need for servlets

Modern web-based systems often require sophisticated and adaptable user interfaces. In the early days of the Web, the main means of making web pages dynamic and interactive were **applets** and **web forms**. We will consider each of these in turn and show their relation to servlets.

Applets

An applet is a block of compiled Java (in bytecode form) that can be downloaded along with a web page and that runs within suitable browsers (Figure 1).

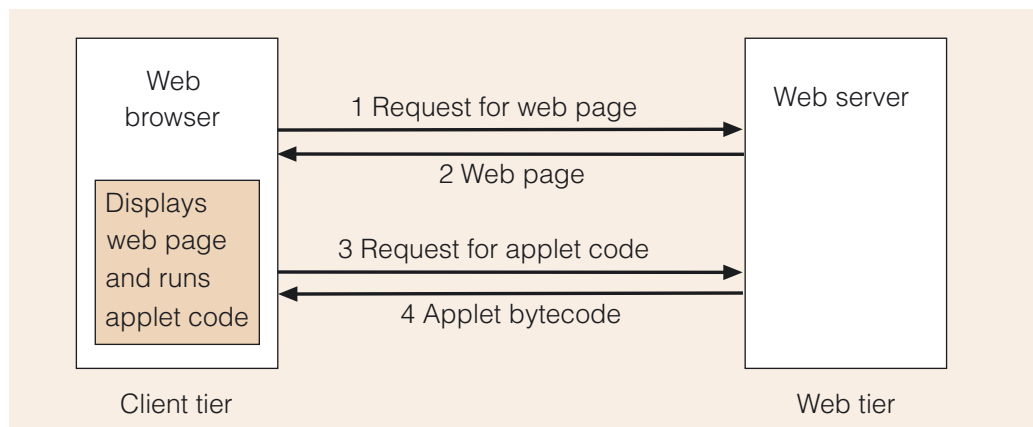


Figure 1 How applets are downloaded and executed – numbers show the order of events

Applets can be used for a wide variety of functions such as complex graphical displays, running simple games, or processing data before sending it to the server. A key feature is that the applet runs within a Java environment provided by the browser and running on the client machine. As applets are automatically downloaded along with their associated web page, by default they are regarded as untrusted – they cannot do anything irreversible on the client machine. **Untrusted applets** are subject to severe security restrictions – for example, they cannot access or change files on the client machine; they can only communicate directly with the server from which they were downloaded. These restrictions can be relaxed for **trusted applets**, which use cryptography to guarantee their origination from a known source. We consider applets in more detail in the next unit, and the associated security issues are discussed in *Unit 10*.

In the early days of the Web and the Java language, there was great excitement among developers about the possibilities of applets for making the Web more dynamic. The two features of applets noted above – the security restrictions and the need to download the applet code from the server each time it runs – soon dampened this enthusiasm among serious developers of distributed systems. The security restrictions made it hard for applets to access remote systems, such as database servers, and the need to wait for

downloading tended to limit severely the feasible size of applet code. One solution is to carry out most of the processing on the server, using servlets.

A servlet is a module of code, stored and run on a server, that follows the **request-response** programming model. Each request from a client elicits a response from the servlet – the exact form of the request and response is defined by the **protocol** in use. In general, servlets can be written to operate under any request-response protocol and they need not be written in Java. In this course, we will be concerned only with HTTP Java servlets, which respond to HTTP requests as sent by web browsers.

We explain how HTTP works in Subsection 2.3.

Since servlets are stored and run in a controlled environment on the server, they do not need to have the severe security restrictions applied to untrusted applets. Servlets can use the full power of the Java language and its libraries – they can access other servers such as database servers and can invoke other programs running on the server hosting the servlet (or ones running on a different server). Also, it is possible to scale up the system by providing additional servers to run more servlets if there is heavy demand on the system, or to provide backup in case of server failure.

Web forms

The other traditional way of allowing some interactivity in web-based interfaces is to use web forms. Most people will have used these when registering or logging in to a web-based application, such as that shown in Figure 2.

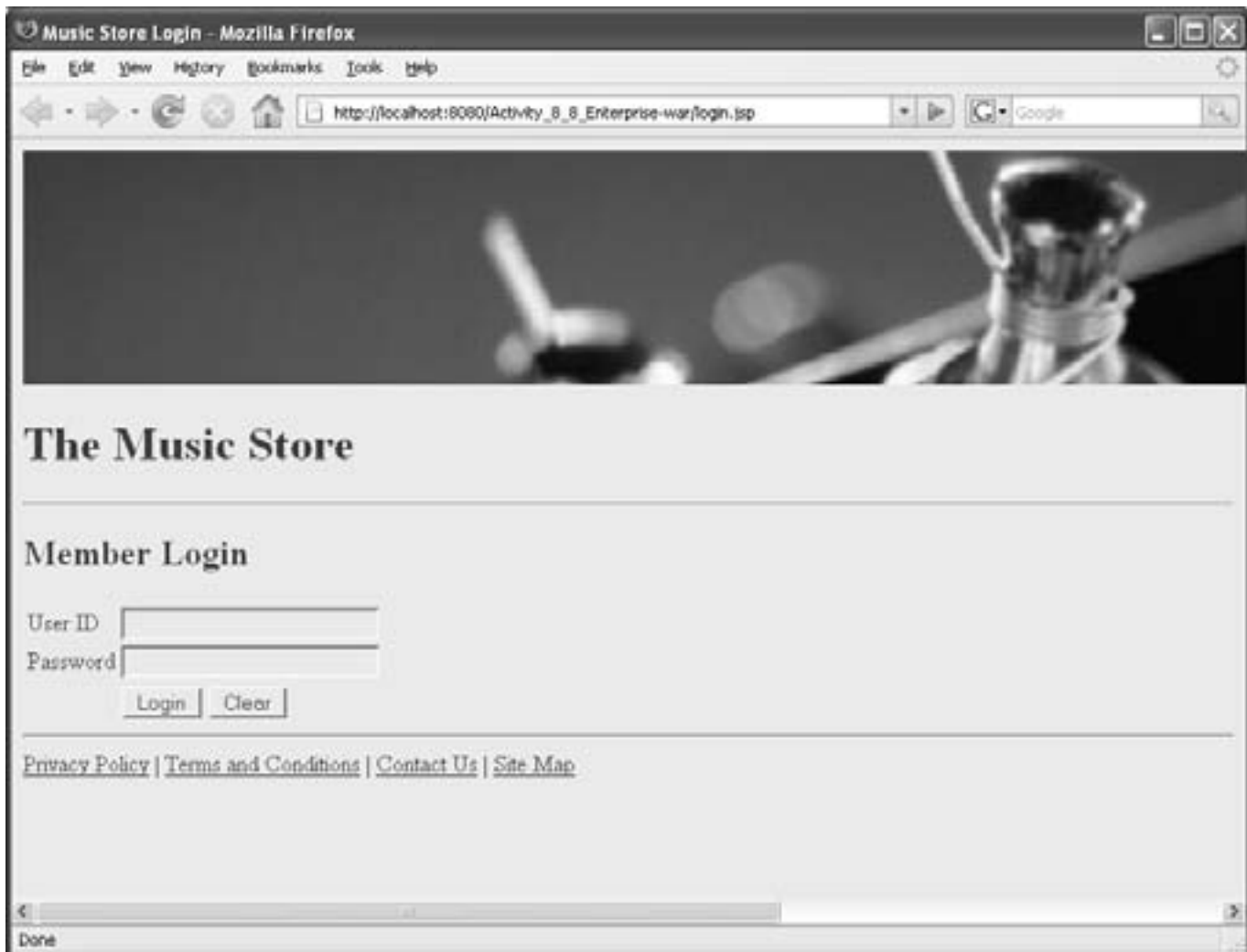
A screenshot of a Mozilla Firefox browser window titled "Music Store Login - Mozilla Firefox". The address bar shows "http://localhost:8080/Activity_8_8_Enterprise-war/login.jsp". The page features a header image of a musical instrument, possibly a trumpet. Below the image, the text "The Music Store" is displayed in a large, bold, serif font. Underneath, the section "Member Login" is shown. It contains two input fields: "User ID" and "Password". Below these fields are two buttons: "Login" and "Clear". At the bottom of the page, there are four links: "Privacy Policy", "Terms and Conditions", "Contact Us", and "Site Map". The browser's status bar at the bottom indicates "Done".

Figure 2 Example of a web form used for logging in to the musical instrument store

When the user has filled in the fields on this form with details of their user ID and password and then clicked the button marked **Login**, the details that have been entered are sent to the server for processing (Figure 3).

Until the invention of servlets, this processing of form data was normally carried out by a **Common Gateway Interface (CGI)** program or CGI script. The CGI is a standard for interfacing external applications with information servers, such as HTTP or web servers. CGI programs are stored on the web server and are run by that server in response to particular HTTP requests, as illustrated in Figure 3. The programs or scripts can be written in any of a number of compiled languages, such as C, C++ and Java, or in scripting languages such as Perl.



Common Gateway Interface (CGI).

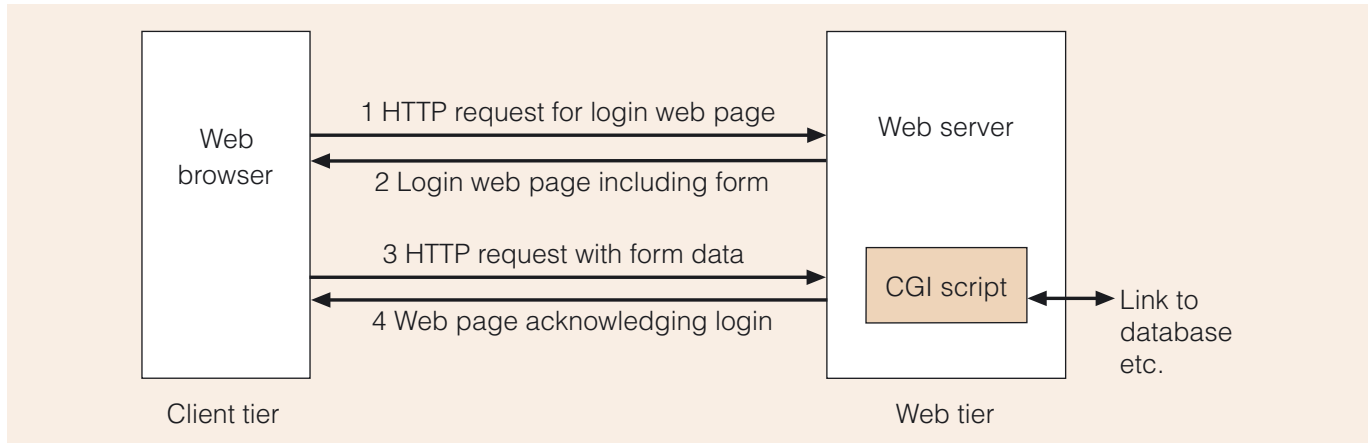


Figure 3 Using a CGI script to process a valid login request from a web form

HTTP is a **stateless protocol** – that is, no information about the request is stored, and each HTTP request is independent of any others. In particular, there is no continuity between subsequent requests from the same user. This can cause difficulties in situations like the ecommerce scenario in Figure 3. After logging in, a user may be presented with a list of items for sale. They may wish to select a number of items to purchase and this will generate a sequence of requests for the server to process. You will recall from the previous unit that this conversation of linked requests and replies between the client and server on behalf of a particular user is called a **session**. It would be very useful if the server could store state information to keep track of all the data related to each session.

CGI scripts are still widely used but they also have a number of drawbacks. Each CGI script or program runs only in response to an HTTP request and then it closes down. It runs in a separate process on the server which must be started up and then shut down for each HTTP request that invokes a CGI script. This gives rise to a considerable overhead in servicing a sequence of requests.

Servlets overcome many of these drawbacks. Once they are started up they continue running on the server and can deal with a succession of HTTP requests, possibly from multiple clients. They use threads rather than processes to service any concurrent requests, and threads are normally much more efficient than processes. They can store the state of a session and can maintain links to resources like a database without having to open and close this link for every request as a CGI script would.

Against these advantages, it is possible that some CGI programs written in compiled languages like C may run faster than a Java servlet – they may therefore still be useful in some relatively simple applications. Although servlets can have advantages in the maintenance of state information, we will see that this must be carefully handled to ensure that any shared data is properly protected.

2.2 How servlets run

The servlet is the foundation of all the technologies that form part of the web tier. Servlets run in an environment known as a **web container**, which provides a number of services including security, concurrency and managing the servlet lifecycle. This means that you do not need to explicitly program many aspects of security and concurrency in the servlet code, as the web container environment will deal with these. In general, a web container may be running several different servlets or several instances of a particular servlet at once. For example, it is often convenient to structure a system into a number of servlets which cooperate to produce the responses to client requests (see Figure 4).

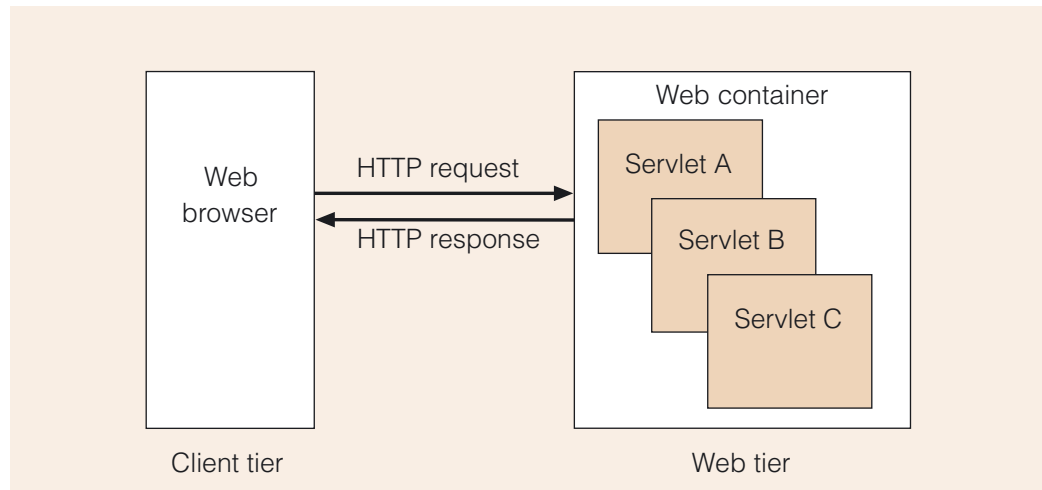


Figure 4 Web container running a number of cooperating servlets

The normal situation is that a servlet is expected to handle concurrent requests, a so-called **multithreaded servlet** (see Figure 5).

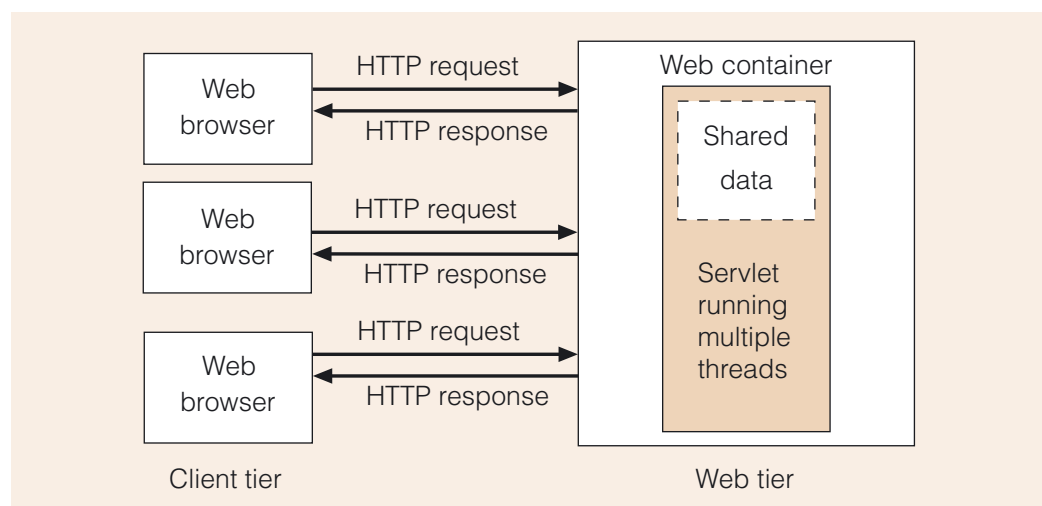


Figure 5 Web container running a multithreaded servlet

Multithreaded servlet code must make sure that any shared resources such as instance variables, class-wide data, files or network connections are properly handled. For example, several users could be, simultaneously, attempting to make changes to a particular file that the servlet has opened on the server – the code must guarantee that any such changes are made in a controlled fashion to avoid corrupting the data. We will see how this works later in this unit.

It is possible to specify that each servlet can handle only one request at a time (a **single-threaded servlet**). In this case, the web container will typically run a separate instance of the servlet for each concurrent request (see Figure 6). Alternatively, it may

restrict concurrent access to the servlet in order to protect shared resources. Clearly, either of these approaches would tend to limit the responsiveness of the system.

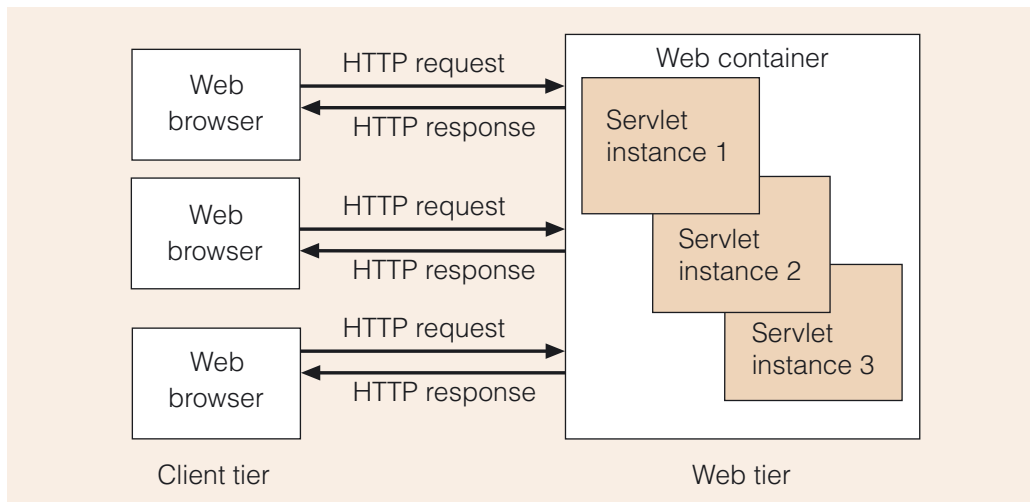


Figure 6 Web container running several instances of a single-threaded servlet

This single-threaded servlet approach does not completely protect against possible interference between concurrent requests so is now not favoured. It may still work in simple applications, but does not scale up well. The preferred approach is to use the normal multithreaded servlet with appropriate handling of any shared resources, as explained later, in Subsection 2.5.

2.3 How servlets are programmed

A Java servlet is an object of a class that implements the `Servlet` interface. This interface, from the `javax.servlet` package, defines a number of **lifecycle methods** required from all Java servlets – see Table 1.

Table 1 Servlet lifecycle methods

Servlet method	Description
<code>init</code>	Allows initialisation of the servlet when it is first executed.
<code>service</code>	Handles any requests from clients of the servlet.
<code>destroy</code>	Allows clean-up or release of resources just before the servlet terminates.

For example, the `init` method could be used to assign values to instance variables, or to open a connection to a network link or to a database. The `destroy` method would typically be used to close any such connections. The `service` method has two parameters corresponding to the request details and the response respectively – its role is to construct an appropriate response object to be returned to the client that sent the request.

Important and specific kinds of servlet are those that respond to the HTTP protocol. In this course, we will be concerned only with HTTP servlets. For these, Java provides an abstract class `HttpServlet`, which implements the lifecycle methods of Table 1. This class can be extended to override the lifecycle methods if required, but the more common requirement is to define appropriate responses to HTTP requests such as GET and POST.

The `service` method of the abstract class `HttpServlet` has a number of helper methods, each corresponding to one of the possible HTTP requests. Table 2 shows a list of these methods, all of which are declared as `protected` and `void`.

Table 2 Servlet service methods defined for HTTP/1.1

HTTP method	Java method	Description
OPTIONS	<code>doOptions</code>	Indicates which HTTP requests this server can respond to.
GET	<code>doGet</code>	Requests a resource, for example a web page, from the server – the request may include a limited amount of parameter data. Both the response header and response body (e.g. the web page) are returned.
HEAD	<code>doGet</code>	A HEAD request is similar to a GET request except that it requests only information about a resource – only the response header information is returned, no response body. Hence it is also handled by the <code>doGet</code> method – useful for checking for valid URLs.
POST	<code>doPost</code>	Allows a block of data to be submitted to the server for processing, e.g. the contents of a web form, or a message to a bulletin board. Typically used for changing the stored state of the system on the server.
PUT	<code>doPut</code>	Stores a specified resource (such as a file) on the server.
DELETE	<code>doDelete</code>	Deletes a specified resource (such as a file) from the server.
TRACE	<code>doTrace</code>	Reflects the request message back to its sender – used for testing or troubleshooting.

To write an HTTP servlet, we define a class that inherits from `HttpServlet`, and override any of the lifecycle or service methods (listed in Tables 1 and 2) where changes from the default are required. There is no need to override all the service methods because the `HttpServlet` class, despite being abstract, defines default implementations for all these methods. In general, these default implementations are not very useful (just returning an error status to the client), but they do fulfil the compilation requirements. The exception is the `doOptions` method, which is fully implemented by `HttpServlet`. Many servlets override only the `doGet` and `doPost` methods (although it is also quite common to override the `init` and `destroy` methods).

When a servlet receives an HTTP request, it invokes the `service` method and passes it references to two objects of classes `HttpServletRequest` and `HttpServletResponse` respectively. The `HttpServletRequest` object contains details of the HTTP request, including the HTTP method name and any associated parameters or other data. The `HttpServletResponse` object is used to construct the response to be returned to the client. The `service` method then invokes the appropriate helper method from those listed in Table 2. This process is illustrated in Figure 7.

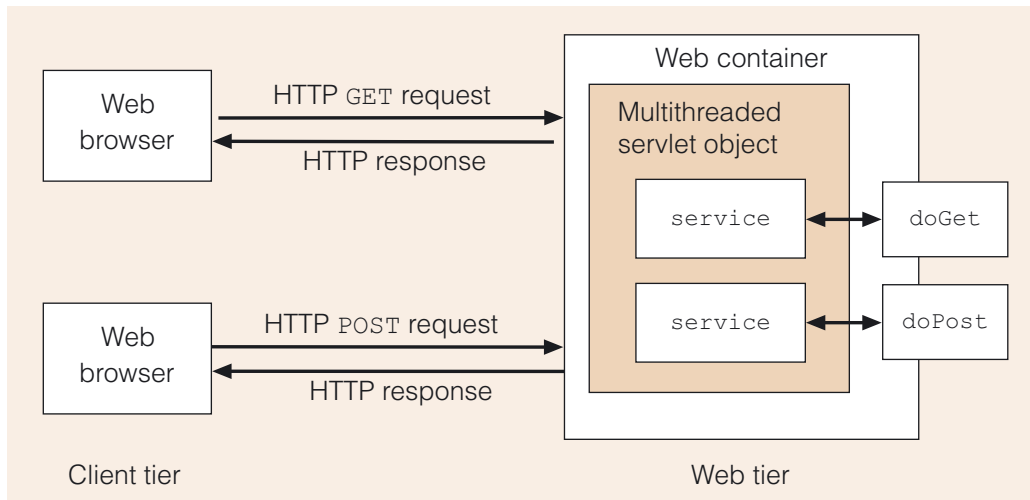


Figure 7 Threads running the `service` method in response to GET and POST requests

When a multithreaded servlet receives a number of concurrent HTTP requests, it starts a separate thread to handle each request. Each thread runs the `service` method, which will invoke the relevant helper method as in Figure 7. It is important to note that this multithreading is handled by the web container (the software environment for the servlet) and does not need to be explicitly programmed by the writer of the servlet. However, the servlet programmer must be aware that this multithreading can occur and should ensure that any data or other resources that might be shared between threads is correctly handled. This is explained in more detail in Subsection 2.5.

The following code shows an example of a simple servlet class, `HelloServlet`, that responds to a GET request by returning a web page with a fixed greeting.

Servlet Example 1 – Returning a simple fixed greeting

```

public class HelloServlet1 extends HttpServlet
{
    protected void doGet(HttpServletRequest req,
                          HttpServletResponse resp)
        throws ServletException, IOException
    {
        resp.setContentType("text/HTML");
        PrintWriter out = resp.getWriter();
        out.println("<HTML>");
        out.println("<HEAD>");
        out.println("<TITLE>Hello Client</TITLE>");
        out.println("</HEAD>");

        out.println("<BODY>");
        out.println("<H1>Greetings Page</H1>");
        out.println("<BR><BR><BR>");
        out.println("<H2>Hello Client</H2>");
        out.println("</BODY>");
        out.println("</HTML>");

        out.flush();
        out.close();
    }
}
  
```

When run, this servlet generates a web page with the heading 'Greetings Page' in large font and the message 'Hello Client' in slightly smaller font. Clearly `HelloServlet1` is a rather complex replacement for a standard web page defined in HTML. Normally, we use servlets to allow us to generate different response pages, depending on some data that accompanies the request, or else to return a page of dynamically generated information, such as the latest contents of a database.

Note that `HelloServlet1` is a fully working servlet – we need to define only the `doGet` method as long as we are not intending to respond to other HTTP requests. As discussed earlier, the parent class `HttpServlet`, although abstract, implements the `service` method and provides default implementations of its helper methods, hence satisfying the compilation requirements.

The following example shows how to extract and respond to some information entered into a web form, such as the login form for The Music Store shown earlier in Figure 2.

Servlet Example 2 – Returning a greeting in response to a user ID

```
public class HelloServlet2 extends HttpServlet
{
    protected void doGet(HttpServletRequest req,
                          HttpServletResponse resp)
                          throws ServletException, IOException
    {
        resp.setContentType("text/HTML");
        PrintWriter out = resp.getWriter();
        out.println("<HTML>");
        out.println("<HEAD>");
        out.println("<TITLE>Hello Client</TITLE>");
        out.println("</HEAD>");

        out.println("<BODY>");
        out.println("<H1>Greetings Page</H1>");
        out.println("<BR><BR><BR>");
        // Store user id value sent as part of form data
        // in the request object.
        String userID = req.getParameter("userid");
        out.println("<H2> Hello " + userID + " </H2>");
        out.println("</BODY>");
        out.println("</HTML>");

        out.flush();
        out.close();
    }
}
```

In this example, the user enters their user ID into a field on the form and clicks on the **Login** button. This causes a GET request to be sent to the server along with the user ID data as a **parameter**. We defer to the next unit the details of web forms, including how parameters are passed to the server. At this stage we show only how to access parameters. Each parameter on a form has a name (`userid` in this case) and this is used to specify the data to be extracted from the `HttpServletRequest` object referenced by `req`, as follows.

```
String userID = req.getParameter("userid");
```

Here the variable `userID` is just a standard local variable in the method `doGet`. An important point here is that variables declared local to servlet methods are

thread-safe – they are not shared between any concurrently running threads and so are safe from accidental corruption. See later in this section for more about this topic.

Servlet Example 3 – Adding to an online shopping cart

The following servlet shows an item (in this case a musical instrument) being added to the list of items in an online shopping cart. The unique identifier of this particular instrument has been entered or selected from a web form, and passed to this servlet as part of an HTTP `POST` request. It is loosely based on the examples from *Unit 7* which used annotations to access EJBs, but is somewhat simplified. At this stage we are focusing not on how the EJBs are accessed or used, but mainly on how servlets can accept requests and associated data and process these. You will see more complete examples later in this unit and in the associated activities.

```
public class AddToCart extends HttpServlet
{
    @EJB
    private InstrumentFacadeLocal instrumentFacade;

    @EJB
    private CartLocal shoppingCart;

    protected void doPost(HttpServletRequest req,
                           HttpServletResponse resp)
        throws ServletException, IOException
    {
        // Get the instrument id from the POST request.
        String instrId = req.getParameter("ID");

        // Find the details of this instrument from the database.
        Instrument instr = instrumentFacade.find(new Integer(instrId));

        // Add the instrument to the cart. Returns true if successful.
        boolean wasAdded = shoppingCart.addToCart(instr);

        if (wasAdded)
        {
            // Mark this instrument as reserved for sale.
            instr.setSold("reserved");
        }
    }
}
```

The most important aspects of this example are as follows.

- ▶ When carrying out actions like this where we are updating server data, such as the shopping cart or the database holding details of the stock of instruments, a `POST` request should be used. `GET` requests should be used only for queries that do not update anything permanently. See Subsection 2.5 for further discussion.
- ▶ The unique ID of the musical instrument to be added to the shopping cart is passed as a request parameter. Access to parameters for `POST` requests (like `ID` above) is by invoking the `getParameter` method, exactly as when handling `GET` requests.
- ▶ There is no requirement for a servlet to produce any output response directly – `POST` requests are often used to invoke some processing, as in this case. We will see later that a servlet may delegate the task of producing a response to another component (another servlet or a JSP page).



Activity 8.1

Running a servlet.



The Java EE API.

2.4 Processing GET and POST requests

The Java EE API places some additional expectations on the behaviour of the HTTP servlet methods that respond to GET and POST requests. The following is an extract from the API specifications for the class `HttpServlet` in the `javax.servlet.http` package.

`doGet`

The GET method should be **safe**, that is, without any side effects for which users are held responsible. For example, most form queries have no side effects. If a client request is intended to change stored data, the request should use some other HTTP method.

The GET method should also be **idempotent**, meaning that it can be safely repeated. The term idempotent derives from Latin *idem* – same and *potent* – powerful or effective, meaning that each execution of the GET method should leave the system in the same state. Sometimes making a method safe also makes it idempotent. For example, repeating queries is both safe and idempotent, but buying a product online or modifying data is neither safe nor idempotent.

`doPost`

This method does not need to be either safe or idempotent. Operations requested through POST can have side effects for which the user can be held accountable, for example, updating stored data or buying items online.

A safe method is defined above as a method having no side effects – this means it does not have any effect on the system state, such as changing stored data. The key point is that GET should normally be used for queries that return information without changing any stored data. This does not prevent `doGet` methods from updating memory variables and so on – it does mean that GET operations should not change data stored in system databases, for example. Operations that may change data in this way should normally use POST.

This distinction in the use of GET and POST is a convention and is not enforced – indeed it may not always be possible to follow these recommendations. For example, if the data to be submitted along with the request is very long (typically, over 2000 characters) or contains non-ASCII characters then it may not be possible to use the GET request (Korpela, 2003). In such cases a POST request should be used instead. This is because of the different ways in which GET and POST requests send data from client to server – this is explained in more detail in the next unit about the client tier.

Due to this somewhat unsatisfactory situation, some developers now write servlets where both GET and POST requests are handled by the same method. For example, the NetBeans IDE automatically generates servlet skeleton code where both the `doGet` and `doPost` methods invoke the same method with the signature:

```
void processRequest(HttpServletRequest req,  
                    HttpServletResponse resp)
```

This is of course relatively straightforward, because accessing request parameters and other data is programmed in exactly the same way for both GET and POST requests.

2.5 Dealing with concurrent requests

Now that we have seen how servlets are programmed, we can return to consider the situation where a servlet may have to respond to multiple client requests concurrently. The standard situation is that of a multithreaded servlet. This means that the web container will create and run a separate thread for the `service` method that is invoked in response to each request. The helper methods such as `doGet` or `doPost` which will be invoked by the `service` method must be written to take account of this (Figure 8).

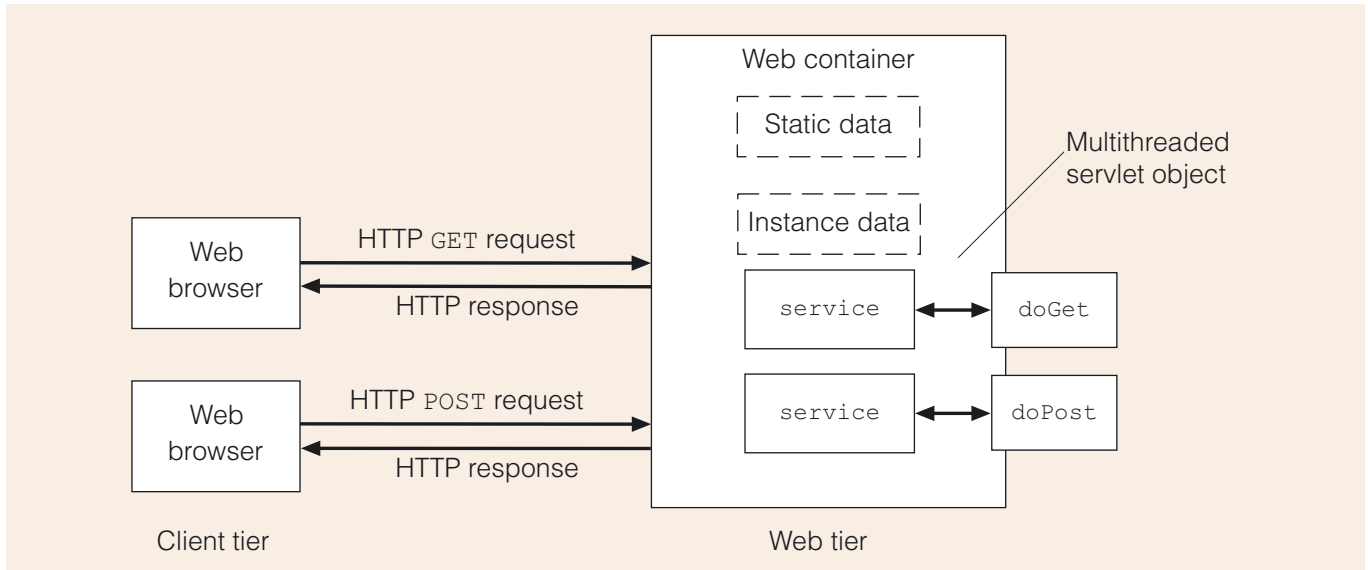


Figure 8 Data shared by multiple service threads responding to concurrent requests

Recall that any data stored in variables declared locally to servlet methods is thread-safe (such as the `userID` variable of the `doGet` method, in Servlet Example 2 above). This is because each thread has its own copy of local variable data.

By contrast, instance variables or static variables are not thread-safe as they are shared between any threads being run concurrently by the servlet. Recall that instance variables are those declared within the servlet class but outside any method. Instance variables can be useful but they must be handled carefully in a multithreaded servlet. Often they are most appropriate for any variable whose value is not known in advance but which must be set once and does not subsequently change. Typically, this is done in the `init` method when the servlet is initialised.

For example, if you wanted to count how many times a servlet had been invoked (for example, to count the number of users who had logged in to a system), then you could use an instance variable that was incremented each time a servlet method was called. If you need to update such instance data, you must synchronise the update code, so that at most one thread at a time can be carrying out an update.

If the servlet accesses external resources (like a database, a file or a network connection), these are normally accessed via an instance variable containing a reference or a handle to the resource. Typically such resources will be opened when the servlet starts up and closed only when the servlet is closed down, using the servlet `init` and `destroy` methods respectively. This avoids the inefficiency associated with CGI scripts, for example. Communication with such external resources must also be protected by synchronisation – otherwise concurrent threads may interleave their output to a file, for example.

It is possible to synchronise the whole `service` method or its helper methods such as `doGet` and `doPost`. While this will protect the shared data, it will usually have an adverse effect on system performance, as it effectively means that HTTP requests can be

processed only sequentially by a given instance of the servlet. It is best to synchronise as little of the servlet code as possible, to maximise performance. So, for example, it may be best just to synchronise the potentially unsafe blocks of code, as in the following example.

```
synchronized(this)
{
    loginCount++;
}
```

If you have to work with servlet code written in some older versions of Java EE, you may encounter another possible approach to concurrent requests, the single-threaded servlet we discussed briefly in Subsection 2.2. We will see that it is not thread-safe and is no longer recommended – we will explain it briefly here as you may encounter it elsewhere and it is useful to understand the issues. In this case, each servlet instance has only one thread of control and so shared resources such as servlet instance variables are distinct for each instance (Figure 9).

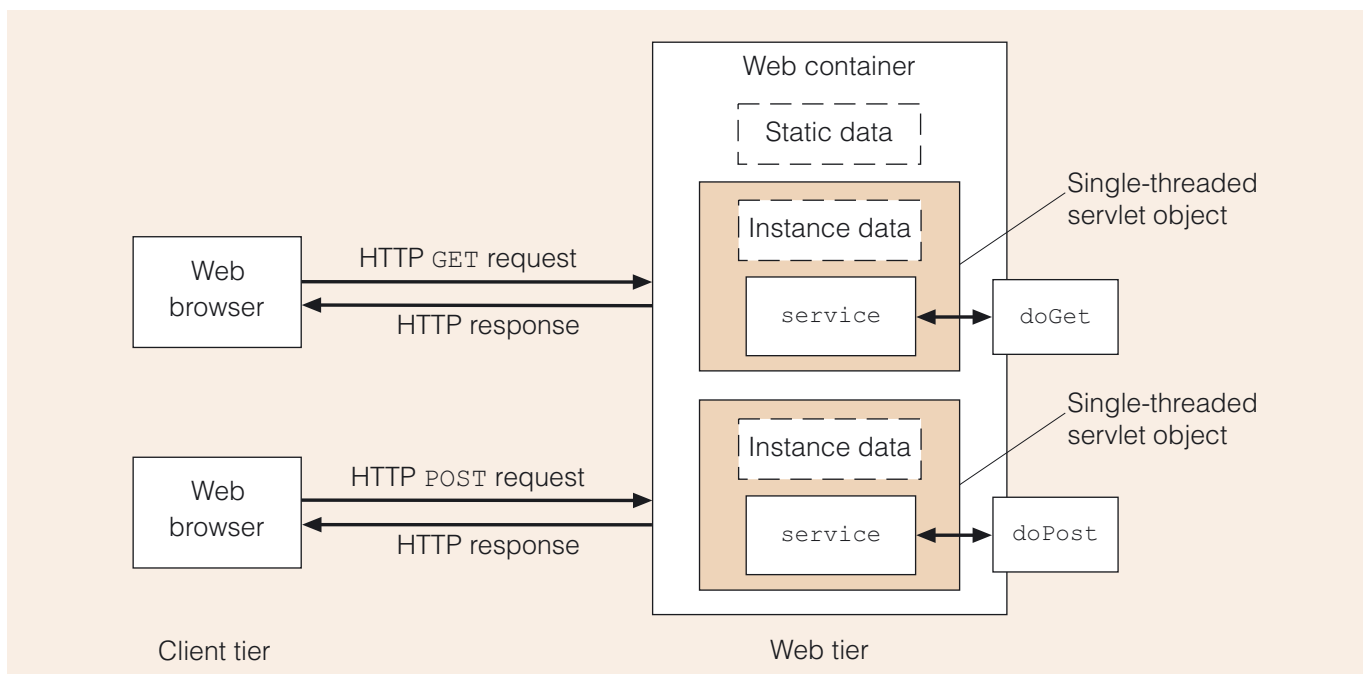


Figure 9 Two servlet objects each with a single service thread responding to one request

This approach can be adopted by declaring the servlet as implementing the interface `SingleThreadModel`. This declaration is all that is required to ensure that the `service` method of this servlet is not executed concurrently by two separate threads. The web container ensures this by either synchronising access to the servlet object or creating a separate instance of the servlet for each new client request (as in Figure 9). It may use a mixture of these two strategies, adjusting to the current demand from clients as required.

The `SingleThreadModel` interface guarantees only that no two threads will concurrently execute the `service` method within the same servlet instance. Since the web container may instantiate multiple instances of a single-threaded servlet class to handle a heavy request load, this approach is not completely safe. For example, it will not prevent potentially undesirable interference between multiple instances of a servlet that share static (i.e. class-wide) variables or session attributes (explained in the next subsection). For this reason, it has been deprecated, starting from Java EE version 4 (known as J2EE 1.4).

The correct approach is to use the default multithreaded servlet with synchronisation, as appropriate, of any shared resources, as explained above. Web containers are

supposed to run only one instance of each multithreaded servlet, but even if multiple instances of the same servlet do execute concurrently, the synchronisation will protect any shared data.



Activity 8.2

Exploring scope and concurrency in servlets.

SAQ 1

What are the advantages and disadvantages of servlets compared to each of the following in providing dynamic content for web pages?

- (a) Applets
- (b) CGI scripts

ANSWER.....

(a) The advantages and disadvantages of servlets compared to applets are as follows.

- ▶ Applets run on the client machine, and so can be very responsive since there are no network delays.
- ▶ The applet code must be downloaded from a server before running, making it less attractive to write large applets.
- ▶ Servlets are more scalable – additional servers can be used to run more servlets if there is heavy demand on the system, or to provide backup in case of server failure.
- ▶ Untrusted applets have severe security restrictions that limit what they are permitted to do on the client machine – although most of these can be avoided using trusted applets.
- ▶ Since servlets are stored and run in a controlled environment on the server, they do not need to have such security restrictions; they can use the full power of the Java language and its libraries; they can access other servers and invoke other programs.

(b) Each CGI script starts a separate process in response to an HTTP request and closes the process after responding. Servlets are more efficient in the following ways.

- ▶ Once started up they continue running on the server and can deal with a succession of HTTP requests, possibly from multiple clients.
- ▶ They use threads rather than processes to service any concurrent requests.
- ▶ They make it easy to store the state of a session and to maintain links to resources like a database.

2.6 Using sessions with servlets

We have seen that HTTP is a stateless protocol – each HTTP request is effectively independent of any previous or subsequent requests. Sometimes this is not a problem, or may even be useful – it is appropriate for the original main use of the Web for delivering static documents.

However, users often want to carry out a series of linked operations, a *session*, where each operation gives rise to one or more HTTP requests. A classic example of this is a shopping cart for an ecommerce system where a user may wish to select a series of items to buy, and may also wish to access additional information at times during the process. In such a situation, it is very helpful if the system can somehow store the current state of the interaction between the user and the system – the session state.

Servlets facilitate this by maintaining session objects that can be used to store and retrieve data, with one object for each client session. The session object can also be used as a means for cooperating servlets or JSP pages within a given session to exchange data.

The session object is of type `HttpSession` and is accessed by invoking the `getSession` method of a request object. For example:

```
HttpSession currentSession = req.getSession();
```

Note that you must call the `getSession` method *before* writing any data to the response – the reason for this will be explained later when we discuss cookies. If the session object does not already exist, then this method will create a new session object.

To store data in the session object, we need the `setAttribute` method, which associates a string name with an object to be stored, as follows.

```
Cart shoppingCart;  
...  
HttpSession currentSession = req.getSession();  
currentSession.setAttribute("cart", shoppingCart);
```

Data stored in the session object can be accessed by passing the name of the required data to the `getAttribute` method of the session object, for example:

```
Cart shoppingCart = (Cart)currentSession.getAttribute("cart");
```

If you are familiar with hash tables or hash maps, such as the Java `HashMap` class, you should recognise this approach of storing data as a key–value pair. In this case, the string name is the key and the object reference is the value. The `getAttribute` method returns a reference of type `Object`, so it is necessary to cast the returned reference to the more specific class `Cart`.

Any data stored in the session object is potentially available to every component (servlets, JSP pages and so on) in the application *for a given client*. Hence, you need to consider synchronising access to the session data, particularly when updating.

2.7 Cookies and sessions

A **cookie** (or web cookie) is an item of text that a web server may send to a client web browser as part of its response to an HTTP request. The web browser stores the cookie text, usually in a small text file. On any subsequent requests from this client, the browser includes the cookie in the information sent to the server. This has many uses, such as making it easier for users to log in to a site they have visited previously, as the text in the cookie identifies a returning visitor to that site. At present we are mainly interested in the use of cookies to maintain an HTTP session.

The standard way for a web application to maintain session information is to send a **session ID** (a unique identifying number or string) in a cookie to the client. The client stores the cookie as explained above. On any subsequent requests from the client, the cookie is sent back to the server, which can extract the session ID, and hence access the appropriate session data for that client session. This is handled automatically by the application server, and the servlet or JSP programmer does not need to do anything special to ensure that session information is maintained. Cookies used for session information are normally deleted from the client storage at the end of the session. This use of cookies is illustrated in Figure 10.

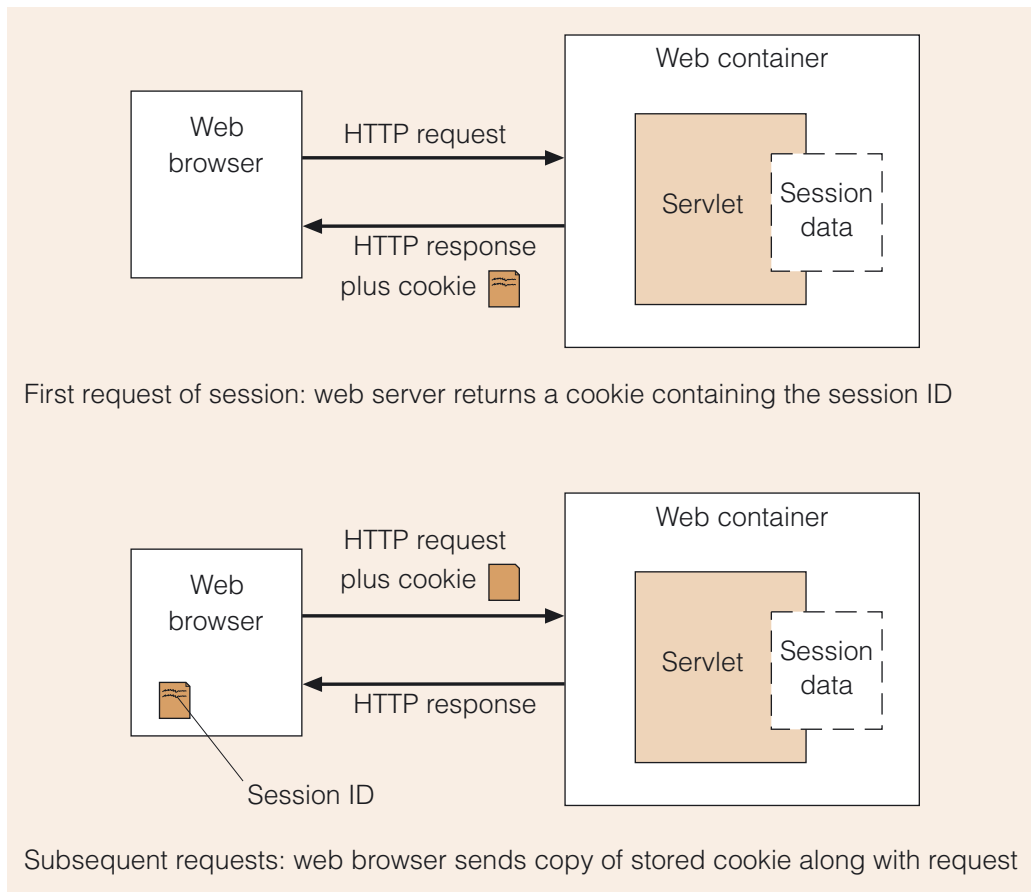


Figure 10 Using a cookie to maintain session information

Cookies can be persistent, meaning that they are stored for a specified period of time, which may extend well beyond a single session – this is done by the server specifying an expiry date when the cookie is created. Cookies are not dangerous in the way that viruses or worms can be, since they are simply a string of text stored on the client computer. However, some users do not trust them, as they can be used for tracking users' behaviour. Hence, most browsers allow users to decide whether to accept cookies – if cookies are not allowed then websites that rely on cookies for session information may not be usable.

However, there is a standard alternative which servers can use if clients are not accepting cookies. Typically any web page will contain a number of links to other web pages or resources, and for each link the underlying HTML takes the form of a URL. In this case, when the server is constructing a web page (the response to a request), the session ID is sent to the client by attaching it to any URLs in the response page. For example, if clicking on a particular link in the web page would normally lead to the URL `http://www.M362musicstore.com`, then a modified URL with a session ID of `ca950fa32f533` would look as follows.

```
http://www.M362musicstore.com;jsessionid=ca950fa32f533
```

The session ID can then be extracted by the server from any subsequent client requests and used to access the session data in a similar way to Figure 10. This approach is a little more cumbersome than using cookies and has the major disadvantage that it needs to be explicitly programmed in the servlet code. To simplify the process a little, the `HttpServletResponse` class has an `encodeURL` method which, if invoked on the response object of a servlet, will append the current session ID to any supplied URL, for example as follows.

```
String newURL = resp.encodeURL("http://www.M362musicstore.com");
```



Activity 8.3

Exploring servlet sessions.

A session ID is just a long sequence of letters and digits – usually longer than this example, but the key thing is that it is unique for each session.



The Java EE 5 tutorial.

To be more precise, the `encodeURL` method will append the session ID only if cookies are turned off in the requesting client browser – otherwise it will not alter the supplied URL. For more details see, for example, the online *Java EE 5 Tutorial*.

Yet another possible approach for maintaining session information is discussed in *Unit 9*, when we discuss hidden form fields as part of our examination of the client tier.

We can now clarify the unexplained requirement above that you must call the `getSession` method *before* writing any data to the response. This is because any cookies sent to the client are sent as part of the response header – if cookies are used to maintain session state then they must be constructed and sent before any data in the body of the response.

SAQ 2

- (a) How can Java servlets deal with a number of concurrent HTTP requests?
- (b) Which method is run by a servlet in response to each user request, and what is the purpose of this method?
- (c) What issues arise in respect of shared resources, and how can they be dealt with?

ANSWER.....

- (a) A servlet is automatically multithreaded – it creates a new thread for each HTTP request it receives, so it can deal with a number of concurrent requests.
 - (b) Each thread runs the `service` method which in turn invokes a helper method such as `doGet` or `doPost`, corresponding to the type of request being processed.
 - (c) Data stored in instance variables or static variables of the servlet is shared between any active threads, and so special care (such as synchronisation) must be taken if such data is updated, to avoid data corruption. The same precautions apply to any resources such as network links or database connections shared between threads.
-

3 JavaServer Pages

Servlets are a very useful technology for processing on the server. However, it should be apparent, even from the simple examples so far, that they can be a rather cumbersome way of generating dynamic web pages. The examples of servlet code in Subsection 2.3 required many `out.println` commands merely to send some standard HTML to the response stream.

A more convenient approach is to use servlets in combination with JavaServer Pages, usually known by the abbreviation JSP. In this section we explain why JSP is useful and give examples to show how it works. There are similar technologies for other environments, such as Microsoft's Active Server Pages (ASP) and the more recent ASP.NET, which use Visual Basic or other languages instead of Java, but otherwise carry out the same sort of function as JSP.



ASP.NET.

3.1 Motivation for JSP

Even in a dynamically generated web page, much of the content is typically static (i.e. the same every time the page is displayed), so we do not really need Java programming of servlets to define it. A JavaServer Page contains the static content of a web page, written in HTML or XML, for example, interspersed with definitions of any dynamic content, mainly specified by fragments of Java code. You can think of a JSP page as a hybrid between a static web page and a servlet (Figure 11).

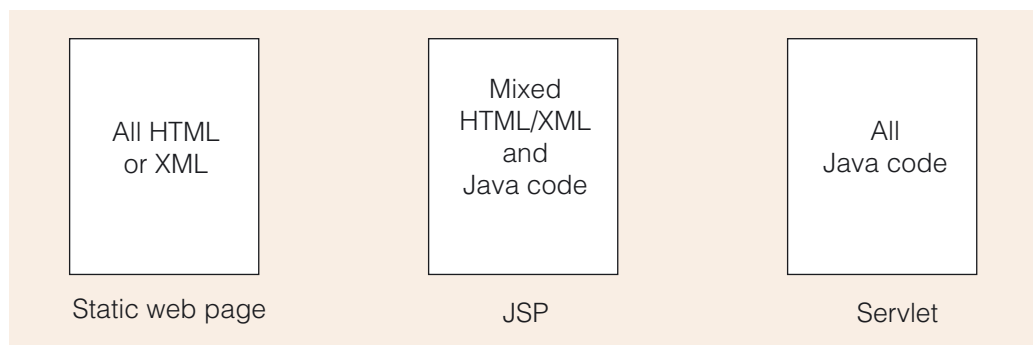


Figure 11 Showing a JSP page as intermediate between a static web page and a servlet

The advantage of this approach is that it allows HTML/XML developers to produce much of the static content for a web page using familiar tools and syntax. It also goes some way to separating the presentation (mainly using JSP) from the processing (mainly done by servlets), which should help with maintenance. We consider this in more detail later when we discuss JSP expression language (JSP EL) and JSP tag libraries.

3.2 JSP syntax

We give here a brief description of the main elements of JSP syntax, but this is not intended to be a comprehensive tutorial. For full details see, for example, *the Java EE 5 Tutorial*.

As a JSP page is a mixture of Java code with HTML or XML, we need a way to make clear which bits of the page are Java code. To do this, we use special **tags** (as in HTML



The Java EE 5 tutorial.

XML (Extensible Markup Language) documents describe the structure of data, unlike HTML which describes how information is presented in a browser. XML is discussed in *Unit 11*.

or XML) to distinguish the fragments of Java code. Anything in the JSP page which is not distinguished by a JSP tag, is normally treated as standard HTML, XML or text and is sent to the browser unchanged. There are three main categories of **JSP tag**: directive, scripting and action.

JSP directive tags

Directive tags provide information to the JSP container about the page, and affect how the JSP compiler generates the servlet that implements the JSP page. There are three directives: `page`, `include` and `taglib`. We will discuss `page` and `include` in this section and we consider `taglib` later, in Subsection 5.1 on tag libraries. Each JSP page can have many instances of these directives. Directives are enclosed in the symbols `<%@` and `%>`.

The `include` directive is used to include another page within the current JSP page. This can be useful to display standard page headers or footers without repeating them on each page. For example:

```
<%@ include file = "std_header.jsp" %>
```

The `page` directive has a number of uses, but one of the most common is to import Java libraries to make them accessible by the Java code within the JSP page, as in the following example.

```
<%@ page import = "java.util.Date" %>
```

JSP scripting tags

Scripting tags are the closest thing to normal Java program elements. There are three types of scripting tag: declaration, scriptlet and expression. Each type uses slightly different enclosing tags. Here is an example of a declaration and initialisation of an instance variable:

```
<%! int userCount = 0; %>
```

A **scriptlet** is a fragment of Java code, consisting of one or more complete Java statements that will be executed when the JSP page runs. For example:

```
<%userCount++; %>
```

A **JSP expression**, as the name suggests, contains a Java expression that produces a value, and this value will be displayed as part of the output. For example:

```
<%= userCount %>
```

JSP action tags

Action tags, as the name suggests, specify some sort of action to be carried out by the JSP page. The JSP specification defines a number of **standard actions** and you can also create your own **custom actions**. Examples of standard actions are including applet code in the JSP page, accessing a JavaBean (see later in this unit), or forwarding control to another JSP page or servlet. For example, the following action passes control to another JSP page to construct the response page.

```
<jsp:forward page = "displayResults.jsp" >
```

We discuss actions in more detail later in Section 4 on JavaBeans and in Subsection 6.1 on how JSP pages and servlets communicate.

JSP tag syntax

There are two main forms of JSP tag syntax. First, there is classic JSP which dates from the earliest versions of JSP. It mainly uses special combinations of symbols like

<%, <%!, <%@ and %> to distinguish JSP elements from text and HTML/XML. As we have seen, a scriptlet looks like this in classic JSP:

```
<% userCount++; %>
```

This classic syntax is not compatible with XML and so there is an alternative JSP tag syntax that follows XML rules and which can be successfully analysed and checked by XML tools. In this XML style, the same scriptlet looks like this:

```
<jsp:scriptlet> userCount++; </jsp:scriptlet>
```

Earlier we saw this example of a JSP directive in classic syntax:

```
<%@ include file = "std_header.jsp" %>
```

In XML-style syntax, it looks like this:

```
<jsp:directive.include file = "std_header.jsp" />
```

In this course, we will normally use the classic JSP syntax for directives and scripting tags, as these are more concise and allow us to set out our JSP code examples more clearly. You will see later that actions and `taglib` directives have only XML-style tags.

A JSP page which uses only XML-style tags is known as a **JSP document**. XML rules mean that each start tag must have a corresponding end tag and that all the JSP elements included in the JSP document must comply with XML syntax. Using JSP documents allows the JSP page to be manipulated and validated by XML tools and can make it easier to automatically generate JSP pages; for example, when several different versions are required to suit different platforms or languages.

3.3 Examples of JSP

JSP Example 1 – Returning the current date and time

```
<%@ page import = "java.util.Date" %>
<%! int userCount = 0; %>
<%! Date now; %>
<HTML>
  <HEAD>
    <TITLE>Hello Time</TITLE>
  </HEAD>

  <BODY>
    <H1>Greetings Page</H1>
    <BR><BR><BR>
    <%
      now = new Date();    // Get current date and time.
      userCount++;
    %>
    <H2>
      Hello, you are visitor number <%= userCount %> <BR>
      The time now is
      <%= now.getHours() %>:
      <%= now.getMinutes() %>:
      <%= now.getSeconds() %>
    </H2>
  </BODY>
</HTML>
```

This example shows the use of a `page` directive to import an item from the Java library package `java.util`. The import syntax within the `page` directive is similar to standard Java syntax – for example, wildcards can be used to import whole library packages, and multiple imports can be listed in a comma-separated list:

```
<%@ page import = "java.util.Date, java.rmi.*, javax.naming.*" %>
```

We have also used a declaration as follows.

```
<%! int userCount = 0; %>
```

This declares and initialises an instance variable called `userCount`, which will record the number of users who have accessed this page. We also declare a variable of the `Date` class, but do not initialise it at this point – this is because we want to get the date and time when the JSP page is called by the user, not the date and time when the JSP page is first invoked and translated.

The use of scriptlets is shown by this code fragment, enclosed in the tags `<%` and `%>`:

```
<%
now = new Date();    // Get current date and time.
userCount++;
%>
```

Java code enclosed in the scriptlet tags is executed, and may set the values of variables, as in this case. In particular it creates a `Date` object that contains the date and time of its creation, and the number of users so far is also updated.

Finally, we show the use of JSP expressions, which are enclosed in tags `<%=` and `%>`:

```
<H2>Hello, you are visitor number <%= userCount %> </H2>
```

Here the actual value stored in the variable `userCount` is sent to the response, so the display would actually show something like:

```
Hello, you are visitor number 42
```

The key difference between scriptlets and JSP expressions is that scriptlets do not normally produce any output directly, whereas the result of a JSP expression automatically appears in the output.

JSP expressions are used again later in the JSP page to display the current time, by accessing the `Date` object referenced by `now`:

```
<%= now.getHours() %>:
<%= now.getMinutes() %>:
<%= now.getSeconds() %>
```

Note that the colons in this case are simply text to be output to separate the hours, minutes and seconds displayed on the web page, for example:

```
The time now is 15:45:6
```

JSP Example 2 – Processing information supplied from a web form

The following JSP page is based on Servlet Example 2 in Subsection 2.3, which responds with a personalised greeting to a user entering their user ID into a web form.

```
<HTML>
  <HEAD>
    <TITLE>Hello Client</TITLE>
  </HEAD>

  <BODY>
    <H1>Greetings Page</H1>
    <BR><BR><BR>
    <%
      // Store user id value sent as part of form data.
      String userID = request.getParameter("userid");
    %>
    <H2>Hello <%= userID %> </H2>
  </BODY>
</HTML>
```

Comparing this example with the servlet `HelloServlet2`, we can see the advantage of JSP – most of the output is just text or HTML and this is added directly to the JSP source. The few items of dynamic content are included using just a scriptlet and a JSP expression. Note that in the JSP page (as with the servlet) `userID` is declared as a local variable rather than an instance variable, to avoid any threading problems.

In this scriptlet we make use of an **implicit object** called `request` – it is implicit in that it does not need to be explicitly declared. It is equivalent to the request parameter of type `HttpServletRequest` passed to the servlet, except that we have no choice about what the JSP object is called. We can invoke the methods of the `HttpServletRequest` interface on this object, in particular the `getParameter` method to give access to parameters such as those entered on a web form. There is a fuller discussion of JSP implicit objects in Subsection 5.2.

Next we look at a more complex JSP example, again relating to The Music Store. Here the database containing the stock of musical instruments is displayed in the form of a table.

JSP Example 3 – Displaying the musical instrument stock list in a table

```
<html>
  <head>
    <title>Display Stock</title>
  </head>
  <body>

    <%
      // Retrieve the stock list from the current session object.
      HttpSession thisSession = request.getSession();
      List allInstruments =
          (List)thisSession.getAttribute("Instruments");
    %>

    <!-- Set up the headings for a table of all instruments. -->
    <table border=1 summary=This table lists all the instruments.>
      <caption><h2>All our instruments</h2></caption>
      <tr><th>Item<th>Category<th>Model<th>Price
        <th>Sold<th>Action</tr>
```



Activity 8.4

Exploring JSP.

```

<%
for (Object o : allInstruments)
{
    Instrument instr = (Instrument) o;
    String instrumentID = instr.getId();
%>
    <tr>
        <td> <%= instrumentID %> </td>
        <td> <%= instr.getModelId().getCategory()%> </td>
        <td> <%= instr.getModelId().getDescription()%> </td>
        <td> <%= instr.getPrice()%> </td>
        <td> <%= instr.getSold()%> </td>
        <td> <a href=AddToCart>Add to cart?ID=instrumentID </a>
                                </td>
    </tr>
<% }%>

<!-- Hyperlink to another JSP page allowing logout. --%>
<br> <br> <br> <a href=logout.jsp>Logout from site</a>
</body>
</html>

```

This JSP page shows much more interleaving of scripting tags with plain text or HTML and may look quite complex. For simplicity and brevity, we have omitted some necessary imports and declarations and there is none of the error checking that would be necessary in production code. It does not matter if you do not follow all the details of this example at this stage – you will have a chance to try out similar examples in the activities later in this unit.

In the first scriptlet, we access the session object in which the list of instruments has previously been stored as an attribute (by some other component of the web tier). The bulk of the JSP page consists of standard HTML table tags – `<th>` for table header, `<tr>` for table row and `<td>` for a table data item (the data in a particular table cell). These are interleaved with JSP expressions that access the data stored in the `Instrument` objects from the `List` object. A standard Java `for` loop enclosed in scriptlets is used to iterate through all the instruments in the list.

The final column of the table is a hyperlink that will invoke a servlet called `AddToCart`. If the user clicks on this link for a particular instrument, it will add the instrument to the online shopping cart, thus reserving it for purchase. Finally, there is a standard HTML anchor tag, used to display a hyperlink that the user can click on to go to a different JSP page to logout from the system. In a more comprehensive system, there would probably be several other alternative links here, offering a choice of actions.

Note the use of JSP comments like:

```

<!-- Set up the headings for a table of all instruments. --%>

```

This comment text does not appear in the response sent to the client – it is just for the benefit of anyone reading the JSP source. You can also use normal Java-style comments within scriptlet code. JSP comments are subtly different in syntax from standard HTML comments like the following, which will show up in the HTML sent in the response:

```

<!-- This is a standard HTML comment. -->

```

The output from this JSP page would be approximately as shown in Figure 12. We have omitted the code for many of the details shown in Figure 12 from the JSP example above. This is in order to keep the example as simple as possible, and to avoid going too much beyond the JSP syntax we have seen so far. The stock list is also rather minimal, which helps fit the figure onto the page.

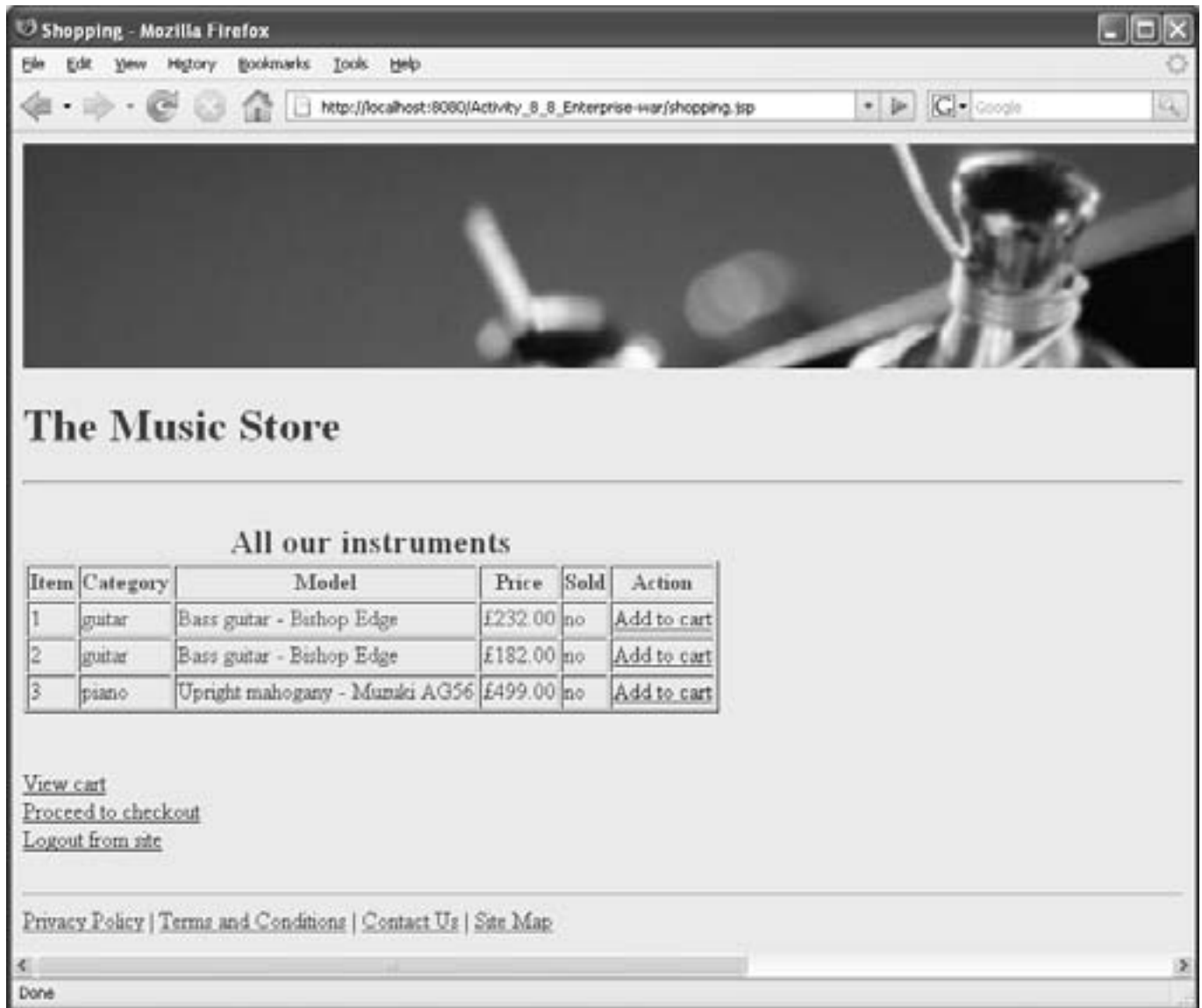


Figure 12 Sample output from JSP Example 3 displaying the instrument stock list

3.4 How JSP works

In practice, JSP is firmly based on servlet technology. We have seen that servlets and JSP offer complementary ways of constructing and displaying dynamic web content. When a JSP page is invoked, it is actually translated to a servlet which runs in a web container in the usual way. So the JSP page shown above for replying to the user login would actually be translated to something like the corresponding Java servlet in Section 2 before being executed (Figure 13).

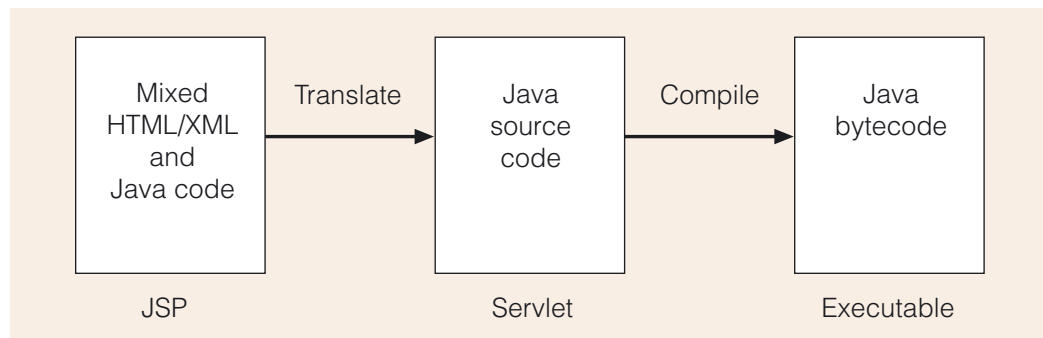


Figure 13 JSP pages are translated to Java servlets before being executed

This translation process has a number of implications.

The first time a JSP page is accessed after being loaded, it must be translated to a servlet and then compiled to bytecode. This servlet bytecode is stored to respond to any further access to that particular JSP page. This process is handled automatically by the web container. Hence, the first access to a JSP page normally takes longer than any subsequent accesses. You should notice this when you try out some of the practical activities that use JSP in this unit and in *Unit 9*.

Because JSP pages are really 'servlets in disguise', many of the issues about concurrency and session management that apply to servlets, apply equally to JSP pages.

SAQ 3

- How do JSP pages differ from Java HTTP servlets?
- What are the main categories of tag used in JSP, and what is each category used for?
- Explain the role of scriptlets in JSP and how they differ from JSP expressions.

ANSWER.....

- Both can produce a response to an HTTP request. A JSP page can contain a mixture of static elements (text, HTML or XML) and dynamic elements (fragments of Java code). A Java HTTP servlet is entirely written in Java and is an instance of a class that inherits from the `HttpServlet` class.
- The categories are directive tags, scripting tags and action tags.
 Directive tags (such as the `page` directive) provide information to the JSP container about the page, and affect how the JSP compiler generates the servlet.
 Scripting tags allow the inclusion of fragments of Java in a JSP page. There are three types of scripting tag: declaration, scriptlet and expression.
 Action tags specify some sort of action to be carried out by the JSP page. They may be standard actions such as including applet code in the JSP page or forwarding control to another JSP page. You can also create your own custom actions.
- A scriptlet is a fragment of Java code, consisting of one or more complete Java statements that will be executed when the JSP page runs. A JSP expression contains a Java expression that produces a value, and this value will be displayed as part of the output.

4

JavaBeans

It is useful at this point to introduce the idea of **JavaBeans** and to distinguish them from the Enterprise JavaBeans components (EJBs) we met in the previous unit. We will then show how they are used in JSP pages and servlets.

4.1 What a JavaBean is

JavaBeans are defined by special Java classes and can be used on any tier of a Java EE system or in ordinary Java applications. They are often useful with servlets and JSP and as GUI components.

Enterprise JavaBeans are, as we have seen, server-based components for Java EE systems, running within an EJB container and used mainly for the business tier of an application.

A JavaBean is an object of a Java class with the following requirements.

- ▶ It has a zero-argument constructor.
- ▶ It has properties which are read/write, read-only, or write-only.
- ▶ Every readable property of the bean has a getter method.
- ▶ Every writeable property of the bean has a setter method.
- ▶ It must be serialisable.

The term **property** in JavaBean terminology loosely corresponds to what Java programmers would call the instance variables of the JavaBean class. Usually the property is implemented directly as an instance variable, but it need not be – the getter method might cause a calculation of a return value rather than a simple retrieval. For example, we could have a property returning a person's age which is calculated from an instance variable that stores their date of birth.

If you read more widely you may encounter some much more complex definitions and requirements for JavaBeans, with talk of visual beans and listener beans – in some situations they are used as components that can be manipulated by special software tools.

In Java EE, most of these more complex requirements are not necessary and the simple requirements above are sufficient for their primary use as a standardised way of storing and retrieving complex data. Because they have a standardised structure it is easy to design software tools, IDEs and JSP tags (as we will see soon) to access them.

The getter and setter methods must follow a standard signature for each property, as follows.

```
public propertyType getTheProperty()  
public void setTheProperty(propertyType theProperty)
```

where *propertyType* means the Java type for that particular property (or instance variable). An exception to this standard form is for `boolean` properties, where the signature of the getter method takes the following form.

```
public boolean isTheProperty()
```

Some people use the term attribute instead of instance variable but we do not want to confuse this with attributes of scope objects introduced later, in Subsection 5.2.



[More on JavaBeans.](#)

The following example of a simple JavaBean class called `UserBean` illustrates these ideas.

To save presentation space we have shown each method on one line – in a source file we would use a more standard layout of several lines per method.

```
public class UserBean implements Serializable
{
    private String userID;
    private String password;
    private boolean privileged = false;
    public UserBean() { }
    public void setUserID(String userid) { this.userID = userid; }
    public String getUserID() { return userID; }
    public void setPassword(String password) { this.password = password; }
    public String getPassword() { return password; }
    public void setPrivileged(boolean priv) { this.privileged = priv; }
    public boolean isPrivileged() { return privileged; }
}
```

Here the class has three properties representing information about the user: the user ID and password, and an indication of whether this user has special privileged access to certain facilities. The default is that users are not privileged. To keep things simple, we ignore password security in this example, and assume that password encryption occurs somewhere else in the system.

4.2 JavaBeans and JSP

JavaBeans are useful in Java EE web tier programming for temporary storage of data, and also for transfer of complex data between web tier components such as servlets and JSP pages.

For example, a `UserBean` object, surprisingly enough, may be just what we need for storing details entered by a user logging in to a system (as in Figure 2). It is important to note that the data will be contained in a Java object held in memory – no persistent storage is carried out.

JavaBeans can be accessed either from servlets or from a JSP page. Using a JavaBean from a servlet is just standard Java programming. In the case of JSP, there are two options – scriptlets or JSP action tags. JSP scriptlets can include exactly the same Java code as used for servlets. However, the recommended, and arguably more elegant, JSP approach is to use the set of standard JSP actions defined for accessing JavaBeans.

We now show examples of some of the relevant JSP tags for JavaBeans. The following `useBean` tag makes a JavaBean called `user` of type `UserBean` available.

```
<jsp:useBean id="user" class="UserBean" />
```

By default its **scope** is limited to the current page, but other scopes are possible – the allowable values are `page`, `request`, `session` and `application`. For example, the following JavaBean has `session` scope, so it continues to exist as long as the client session continues.

```
<jsp:useBean id="user" class="UserBean" scope="session"/>
```

A JavaBean with `request` scope is retained from the time a client sends an HTTP request until the response to that request has been sent. The duration of `application` scope is until the server stops the application – so `application`-scoped beans are available to every session and every request.

All the identifiers in JSP tags are case sensitive – so it's `<jsp:useBean>`, not `<JSP:useBean>`, `useBean` not `usebean`, and so on.

The `useBean` tag creates the JavaBean object if it does not already exist, or accesses a previously created object if there is one associated with the specified `id`.

There are several possible forms of the `setProperty` tag, used to assign a value to a JavaBean property as follows.

```
<jsp:setProperty name="user" property="userID" value="BSmith" />
<jsp:setProperty name="user" property="userID" param="loginID" />
<jsp:setProperty name="user" property="userID" />
```

Note that the value of the `name` attribute in the `setProperty` tags must match the value of the `id` attribute from the `useBean` action that created this JavaBean. These values are case sensitive and it can lead to obscure errors if you ignore this, because several JavaBeans with similar names may be created by different `useBean` actions.

This seems to be an irritating inconsistency but we are stuck with it.

The first of the above `setProperty` tags assigns a specific string value "BSmith" to the property `userID` – in effect, it invokes the `setUserID` method. The value in this case must be a suitable type for the property. The second version sets the same property `userID`, but copies the value from the parameter called `loginID`, extracted from the request object. The final version again sets the property `userID` but this time implicitly copies the value of a request parameter with the same name `userID`. This last version can be very useful as we will see in a more extended example below.

There is also a tag to return the value stored in a property, for example:

```
<jsp:getProperty name="user" property="userID" />
```

4.3 JavaBeans example

We now apply these JSP JavaBeans tags by extending an earlier JSP example.

Assume that the user has completed the form in Figure 2 and pressed the **Login** button. This sends a `GET` request to the server together with values for two form parameters named `userid` and `password`. In an earlier section we have seen how to extract these parameter values from the request object in a servlet. Now we consider how to write a JSP page to extract these values and store them in a JavaBean object.

```
<HTML>
  <HEAD>
    <TITLE>Login Response</TITLE>
  </HEAD>

  <BODY>
    <H1>Login Response</H1>
    <BR><BR><BR>
    <%-- Store user id and password in JavaBean user. --%>
    <jsp:useBean id="user" class="UserBean" scope="session"/>
    <jsp:setProperty name="user" property="userID"
                      param="userid" />
    <jsp:setProperty name="user" property="password"
                      param="password" />

    <%-- Retrieve user id from JavaBean and send to output. --%>
    <H2>Hello <jsp:getProperty name="user" property="userID" /></H2>
    You are now successfully logged in
  </BODY>
</HTML>
```

We can make a couple of refinements to this code. First, since the `password` request parameter has the same name as the `password` property of the `JavaBean`, we can simplify the `setProperty` tag as follows.

```
<jsp:setProperty name="user" property="password" />
```

Note that we cannot do this for the `userID` property since its name is different from the request parameter `userid`, even though only in case. This emphasises that the names must match exactly for the above abbreviated `setProperty` tag to work.

There is one further refinement of the above code that makes `JavaBeans` particularly useful for storing parameter data from HTTP requests. This is due to the use of wildcards. Consider the following form of the `setProperty` tag.

```
<jsp:setProperty name="user" property="*" />
```

This has the effect of setting all the properties in the `JavaBean` `user` to the values of the correspondingly named request parameters. So, for example, the value of the request parameter `userID` is copied to the `JavaBean` property `userID` and so on. Any `JavaBean` properties without a matching request parameter will be left unchanged. The `"*"` symbol in this case is an example of a wildcard.

To take advantage of this wildcard option, we would have to change the request parameter name to match the `JavaBean` property (or the other way round). If we assume that we have done the former, then the JSP page above simplifies to the following.

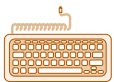
```
<HTML>
  <HEAD>
    <TITLE>Login Response </TITLE>
  </HEAD>

  <BODY>
    <H1>Login Response</H1>
    <BR><BR><BR>
    <%-- Use wildcard to store user id and
      password in JavaBean user. --%>
    <jsp:useBean id="user" class="UserBean" scope="session" />
    <jsp:setProperty name="user" property="*" />

    <%-- Retrieve user id from JavaBean and send to output. --%>
    <H2>Hello <jsp:getProperty name="user" property="userID" /></H2>
    You are now successfully logged in
  </BODY>
</HTML>
```

Note that the `JavaBean` identified by `user` has `session` scope – that means the associated data will be stored as long as the current session lasts.

We can now see why the restrictive structure of `JavaBeans` outlined in Subsection 4.1 is useful in JSP. Because the `JavaBean` constructor has no arguments and because we know each property must have a getter method and a setter method, the JSP tags used in the above examples will work for any `JavaBean`.



Activity 8.5

Using `JavaBeans`.

4.4 JavaBeans and servlets

As mentioned above, JavaBeans can also be accessed from servlets, using standard Java coding. For example, the following code accesses the `userID` property of the same JavaBean named `user` that was shown in the JSP examples above.

```
HttpSession session = req.getSession();
UserBean user = (UserBean)session.getAttribute("user");

String userID = user.getUserID();
```

SAQ 4

What is the difference between a JavaBean and an EJB (Enterprise JavaBeans) component?

ANSWER.....

JavaBeans are defined as special Java classes and can be used on any tier in a Java EE system or in ordinary Java applications. A JavaBean class must have a zero-argument constructor. Every readable property of the bean must have a getter method and every writeable property must have a setter method. In Java EE web tier programming they are useful for temporary storage of data and also for data transfer among servlets and JSP pages.

Enterprise JavaBeans are server-based components for Java EE systems, running within an EJB container and used for the business tier of an application. EJB session beans are used to define the business logic of a Java EE application. They have a business interface that defines their publicly available methods, and this can be configured for either local access (within the same JVM) or remote access (from another JVM, possibly on a remote computer).

5

More on JavaServer Pages

In early versions of JSP, it was common for pages to include significant amounts of Java code (such as expressions and scriptlets) as well as the static content (such as HTML, XML or text). The trend in later versions of JSP (since JSP 2.0) is to minimise direct use of Java code in favour of using **JSP tag libraries** and **JSP expression language** (JSP EL). These essentially have the same role as the Java expressions and scriptlets but use different syntax in an XML style.

The rationale for this is to make it easier to keep processing logic (i.e. Java) separate from presentation information. When JSP pages have complex arrangements of tags interspersed with fragments of Java, they can be hard to read and hence difficult to maintain (consider JSP Example 3 in Subsection 3.3).

Many large organisations have teams of web designers who are separate from their teams of Java developers. JSP tag libraries and JSP EL are supposed to make it easier for web designers who are not Java programmers to produce and maintain JSP pages, leaving the Java programmers to write the code for servlets. See the later section on the model–view–controller (MVC) architecture for more on separating processing from presentation.

5.1 JSP tag libraries

We have already seen quite a few JSP tags, such as those used for directives, scripting and actions. It is also possible for users to define **custom tags**, which are used to carry out certain tasks that would otherwise require Java scriptlets or expressions. For example, the custom tag `<c:set>` assigns a value to a local variable; for example, as follows using a variable called `loggedIn`.

```
<c:set var="loggedIn" value="true" />
```

In practice, most writers of JSP pages will not define their own custom tags but are more likely to use custom tags from a predefined custom tag library. When the JSP page is executed, each custom tag gives rise to some associated Java code, a so-called **tag handler**, but the JSP page writer need never see this code – it is loaded automatically from the tag library.

The generic syntax for custom tags is as follows.

```
<prefix: tagname attribute1="value1" ... attributen="valuen" />
```

Here *prefix* indicates which tag library is being used – the prefix `c` used in the `<c:set>` tag above indicates that we are using the core tag library – this is discussed in more detail below. The number of attributes depends on the particular tag, and there may be no attributes. An alternative form for tags is to have a body between a start and end tag as follows.

```
<prefix: tagname attribute1="value1" ... attributen="valuen" >
    tag body goes here
</prefix: tagname>
```

The most important tag library is the **JSP Standard Tag Library (JSTL)** which, as the name suggests, comes as a standard component of the Java EE environment. In this course we will confine ourselves to the JSTL and will not consider how to define your own specialised tag library – for more details on this see *the Java EE 5 Tutorial*.



The Java EE 5 tutorial.

The JSTL is actually composed of several categories of tag actions, each of which is usually considered a separate library as follows:

- ▶ core actions – dealing with variables, flow control, URLs;
- ▶ XML processing – parsing, flow control, stylesheet transformation for XML documents;
- ▶ internationalisation and text formatting;
- ▶ SQL actions – for database access via JDBC.

The core tag library contains tags that are the equivalent of the standard flow control constructs in Java for selection and repetition. The prefix for the core library is `c`, and the custom tag `if` from the core library can be used as follows.

```
<html>
...
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
...
<c:if test = "${!empty param.userID}" >
    Hello ${param.userID}, welcome to The Music Store website.
</c:if>
...
</html>
```

Note the `taglib` directive, which specifies the URI for the location of the core tag library, and also the prefix to be used to access tags from this library:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

The prefix allows us to use tags from a number of libraries even if they have the same tag name – as this example suggests, you could choose to use a different prefix for the core library. Most of the standard tag libraries have a conventional prefix and it is usually best to follow this convention to avoid confusing other readers of your JSP code. The internationalisation tag library has the standard prefix `fmt` (short for format, as one of its major functions is formatting items appropriately for different countries or languages). The necessary `taglib` directive for this library is as follows.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
```

The other libraries in the JSTL can be accessed using equivalent `taglib` directives – the XML processing tags have the prefix `xml` and the database access tags use the prefix `sql`.

The internationalisation tag library

The internationalisation tag library can be used to make the JSP pages of a web application suitable for use in a variety of locales (countries or language areas). For example, the `<fmt:message>` action can take account of the current locale to display messages in the appropriate language. Each message or phrase is stored using an associated key value to label the text. The text is then retrieved using a tag such as the following, with the key value of "greeting".

```
<fmt:message key="greeting" />
```

The text displayed by the JSP page automatically takes account of the current locale (language, script and so on) so that it might be displayed in the web page as follows.

(In French) Bienvenue au Magasin de Musique!

(In English) Welcome to The Music Store!

There are also tags to allow the resetting of the current locale and many other useful operations.

Returning to the core tag library example above, note the form of the `if` construct with a start tag and end tag enclosing a body as follows.

```
<c:if test = "${!empty param.userID}" >
  Hello ${param.userID}, welcome to The Music Store website.
</c:if>
```

The `<c:if>` tag has one attribute named `test`, which is followed by a Boolean-valued expression. If the expression evaluates to `true`, then the body of the `if` is included in the response, otherwise it is ignored. In this case, if the `test` expression is `true`, the `Hello` message will be displayed as part of the response web page.

You may be a little puzzled by the items enclosed in braces following a `$` sign. We will explain these in more detail in the next subsection – for the moment, you need to know that these are simple examples of JSP expression language (JSP EL). The expression `${param.userID}` gives the value of the request parameter called `userID`, which should have been set earlier, normally by the user completing a web form. If the `userID` parameter has not been set, the expression `${!empty param.userID}` evaluates to `false` and the `Hello` message will not be displayed in the response.

There are many more custom tags in the core library of the JSTL, such as `<c:forEach>` for iteration, `<c:choose>` for selection among a number of possible actions, `<c:set>` to set the value of a variable, and so on. Note that the `<c:if>` tag has no equivalent of the `else` part found in most programming languages – for this you need the `<c:choose>` tag.

Database access tag library

The SQL tags in the JSTL are intended to allow access to databases via JDBC and SQL commands. For example, the `<sql:query>` tag can be used to carry out database queries using the `select` command or other SQL commands.

However, accessing a database directly from a JSP page is actually in conflict with the principle of separating processing from presentation – this is clearly processing. So, the whole SQL tag component of the JSTL has been deprecated – developers are encouraged to use EJBs for database access, except for very simple systems or for short-lived prototypes.

5.2 JSP expression language

Expression language (EL) makes it possible to conveniently write expressions that can access data stored in JavaBeans components and in a number of implicit objects. The latter are objects created by the web container and contain information related to a particular request, page, session or application. For example, we met the implicit object `request` in JSP Example 2 in Subsection 3.3 and this allowed us to extract the `userid` parameter to the HTTP request that invoked this JSP page.

EL can be used as an alternative to accessing these objects using Java language syntax in scriptlets or Java expressions. This is part of the effort to separate presentation from processing and to allow JSP pages to be maintained by web developers who may not be Java experts.

EL expressions are enclosed in the characters `${` and `}`, as in the following example.

```
${staff.age > 21}
```

This accesses the `age` property of a JavaBean called `staff` and returns a Boolean result indicating whether or not the value of the property `age` is greater than 21.

EL has most of the standard operators and literals that are found in Java. The logical and relational operators have textual forms in EL as well as their standard Java forms – for example, the operators `&&`, `||` and `!` can also be expressed as `and`, `or` and `not` respectively; `==`, `!=`, `<`, `<=`, `>` and `>=` correspond exactly to `eq`, `ne`, `lt`, `le`, `gt` and `ge` respectively. The textual forms can be useful in avoiding confusion between the arithmetic operators and various sorts of tag that use the same symbols.

The literal types include Boolean, integer, floating point, string and null, as in Java. EL also has an `empty` operator (not found in Java) – it returns `true` if a value is `null` or empty. For example, this extract from an earlier example shows its use:

```
<c:if test = "${!empty param.userID}" >
```

Implicit objects

Some of the important implicit objects are listed in Table 3.

Table 3 Important implicit objects for JSP

Implicit object	Corresponding class (from <code>javax.servlet</code>)	What it means, and an example of use in JSP EL
<code>param</code>	(not applicable)	Allows access to a request parameter, e.g. <code>\${param.userid}</code>
<code>pageContext</code> (This has four sub-objects as listed below.)	<code>jsp.JspContext</code>	The context for the current JSP page, e.g. <code>\${pageContext.pageCounter}</code> Also allows access to the implicit objects for web application, session, request and response.
<code>servletContext</code>	<code>ServletContext</code>	The web application context for the JSP page's servlet and any web components contained in the same web application, e.g. <code>\${pageContext.servletContext}</code>
<code>session</code>	<code>http.HttpSession</code>	The session object for the current client, accessed as <code>\${pageContext.session}</code>
<code>request</code>	<code>ServletRequest</code>	The request leading to this JSP page, accessed as <code>\${pageContext.request}</code>
<code>response</code>	<code>ServletResponse</code>	The response to be returned to the client – not often used by JSP, e.g. <code>\${pageContext.response}</code>

The `pageContext` and the three objects immediately following it in Table 3 give access to what are known as **scope objects**. Scope is a term meaning the extent over which an object or other item is available. We have already met the session object and the request object in previous servlet examples. For example, an object with session scope continues to exist throughout the duration of a client session, and can potentially be accessed from any component involved in that session. The other two scope objects are the page object, for items whose scope is restricted to the current JSP page only, and the web application scope object, which is accessible by any servlet or JSP page in a given web application throughout the whole time that application is running.

The `response` object (that we met previously in servlet examples) is also accessible from the `pageContext` object, but is seldom used explicitly in JSP code, and is not used as a scope object.

For example, you can use EL to access the implicit `session` object as shown in the following extract, which would display a message including the current session ID.

```
The current session ID is ${pageContext.session.id}
```

The `session` object is an instance of the `HttpSession` class. You can use this syntax to access only the *instance variables* of the `HttpSession` class such as `id`. You cannot use this approach to access the *attributes* of the session object or of other scope objects – recall that we can associate data with scope objects such as the `session` object or the `request` object and this data is described as an attribute of that object. This access to attributes is carried out using the four objects listed in Table 4.

Table 4 Implicit objects for scoped variable access

Implicit object	What it means	Example of use in JSP EL
<code>applicationScope</code>	Maps application-scoped variable names to their values.	<code>\${applicationScope.id}</code>
<code>sessionScope</code>	Maps session-scoped variable names to their values.	<code>\${sessionScope.user}</code>
<code>requestScope</code>	Maps request-scoped variable names to their values.	<code>\${requestScope.quantity}</code>
<code>pageScope</code>	Maps page-scoped variable names to their values.	<code>\${pageScope.counter}</code>

The implicit objects in Table 4 are maps in the same sense as the Java `HashMap` class – a collection of items, each of which associates a key with a value. You may recall in our earlier examples of servlet code that we associate data, such as a `JavaBean`, with the session object using code like this:

```
HttpSession session = req.getSession();
session.setAttribute("user", userObject);
```

The `setAttribute` method adds a key–value pair to the map that holds the session attributes. It associates the string name `"user"` with the `JavaBean` referenced by the variable `userObject`. We can retrieve this data, perhaps in a different servlet, by specifying the key value `"user"` as follows.

```
HttpSession session = req.getSession();
UserBean userObject = (UserBean)session.getAttribute("user");
String userID = userObject.getUserID();
```

Instead of including this Java code as a scriptlet in a JSP page, we can do the same thing in JSP EL by using the implicit `sessionScope` object. For example, to access the `JavaBean` object associated with the current `session` object using the name `"user"`, we simply use the following expression.

```
${sessionScope.user}
```

Note that we use the name `user` here as if it were an attribute of a Java class – there are no quotes. This expression actually returns a reference to the `JavaBean`. Normally we would want to access one of its properties, which we can do simply by adding a dot

followed by the property name – the JSP EL takes care of calling the appropriate getter method, as follows.

```
The user is ${sessionScope.user.userID}
```

Accessing attributes of the other scopes, such as request, page or web application scope, is similar to the above. We return to this topic again in Subsection 6.2 where we consider how servlets and JSP pages can collaborate and exchange data with other servlets and JSP pages within a structured framework.

Scriptless JSP

If you want to strictly enforce the separation of processing from presentation, it is possible to ensure that JSP pages do not accept any Java-style scriptlets. This is done by marking JSP pages as **scriptless**.

This can be specified for some or (more usually) all JSP pages in an application using a **deployment descriptor**. This is an XML file automatically generated by the IDE that defines the internal structure of a Java EE component, module or whole application. A complex application will normally have a number of deployment descriptors corresponding to different tiers or components of the application. As well as listing all the files that make up the component it describes, it also allows the setting of a number of attributes of the item it describes, such as transaction behaviour or security settings.

In general, manipulation of Java EE deployment descriptors is outside of the scope of this course, and many of its functions in setting attributes are more conveniently handled by annotations, as we saw in *Unit 7* when considering transactions. Some IDEs, including NetBeans, offer a convenient way to edit the deployment descriptor without having to get directly involved in editing the XML.

For the web tier of Java EE, the deployment descriptor is normally called `web.xml` and is located in folder `WEB-INF` of the web component of an application. As well as listing all the servlet and JSP files in the web tier, it also allows setting of various attributes by means of XML tags.

For our purposes, we just need to note that some, or all, of an application can be declared as scriptless – any examples of JSP scripting (scriptlets, declarations or expressions) will then cause a compilation error. This can be useful in large systems in identifying any legacy JSP code that is using scripting even though the development standards require use of tags and EL instead.

JSP Example 4

This example extends the JavaBean example in Subsection 4.3 to demonstrate use of JSP EL and custom tags.

```
<HTML>
  <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
  <HEAD>
    <TITLE>Login Response </TITLE>
  </HEAD>

  <BODY>
    <H1>The Music Store </H1>
    <BR><BR><BR><BR>
```

```

<H2>
  <c:if test = "${!empty param.userID}" >
    <%-- Use wildcard to store user id
        and password in JavaBean. --%>
    <jsp:useBean id="user" class="UserBean" scope="session" />
    <jsp:setProperty name="user" property="*" />

    <%-- Retrieve user id from JavaBean
        and send to output. --%>
    Hello ${user.userID} <br>
    <c:if test = "${!empty user.password}">
      <c:set var ="loggedIn" value = "true" />
      You are now successfully logged in <br>
    </c:if>
  </c:if>
  Welcome to The Music Store website!
</H2>
</BODY>
</HTML>

```

In this example, the user should log in by entering a user ID and password into a form, before clicking a button to send these to the server. This JSP page checks that both the user ID and the password are not empty before announcing to the user that they are successfully logged in. This is, of course, a rather low standard of security, but it does show the use of the `<c:if>` and `<c:set>` custom tags, together with JSP expression language, to access both form parameters `${param.userID}` and JavaBean attributes such as `${user.password}`. If the user has not completed the login details they will still receive a welcome message but will not be greeted by name (or user ID).

SAQ 5

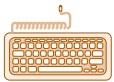
What are the main reasons suggested for using JSTL tags and JSP expression language in JSP pages in preference to Java scriptlets and expressions?

ANSWER.....

There are two main arguments for this.

First, it helps to separate processing (Java code in servlets) from presentation (JSP pages using tags and EL). This may help maintenance in complex applications – for example, presentation aspects could be changed without any accidental changes to processing if the two are adequately separated. It can also be hard to read source files containing complex interleaving of tags and Java code.

Second, it allows web designers familiar with HTML and XML, but not with Java, to produce and maintain JSP pages without any Java programming.



Activity 8.6

JSP EL and custom tags.

6

Structuring the web tier

Typically the web tier for a substantial system requires a combination of servlets, JSP pages and possibly other components such as JavaBeans. Each component should be used for what it does best. Servlets are suited to processing and JSP is best for presentation of dynamic web pages. In this section we will consider how to structure the web tier components.

6.1 Model 1 and Model 2 approaches

In many of the examples presented so far in this unit, the structure of the servlets and JSP pages has been rather ad hoc. Each component may have included a mixture of control logic, presentation logic and sometimes business logic. This rather unstructured approach to web applications is sometimes dignified by calling it the **Model 1** approach. It is a reasonable approach for small-scale simple applications.

However, the Model 1 approach does not scale up very well to large applications that require many cooperating servlets, JSP pages and business tier components. In this case it makes sense to have some sort of division of labour between the various components so that each one is used for the sorts of operation it is most suited to.

The most often recommended approach for structuring a web application is the **model-view-controller (MVC)** pattern. This has been around for a long time, and is most famously associated with the GUI-based systems in the Smalltalk language which started in the 1970s (Burbeck, 1992). However, it is a design pattern and, as such, is independent of particular languages or platforms.

MVC defines three categories of components: model, view and controller. The model represents the system data, such as a shopping cart in an ecommerce system. The view allows the state of the model to become visible – it is the output part of the user interface, such as the HTML pages in a web application. The controller manages the communications between model and view in order to respond to user requests or other events.

You may not be too surprised to learn that, in web applications, MVC is sometimes called the **Model 2** approach. In the web applications that we have seen, we can identify the model with the data, which is typically associated either with the business tier components (such as EJBs) or with JavaBeans. The view is normally the role of JSP pages, which handle the construction of dynamic web pages. The controller is normally implemented by one or more servlets that coordinate the operation and communication of the other components. You may find some variation among authors and practitioners in the precise definitions and implementation of each category, but the basic idea of separating these three aspects of an application is quite widespread and useful.

In a simple MVC approach, there will be a controlling servlet (see Figure 14). This allows central control of security, logging and decisions on what to display next. In more complex systems there may be several, or even many, cooperating servlets forming the controller.

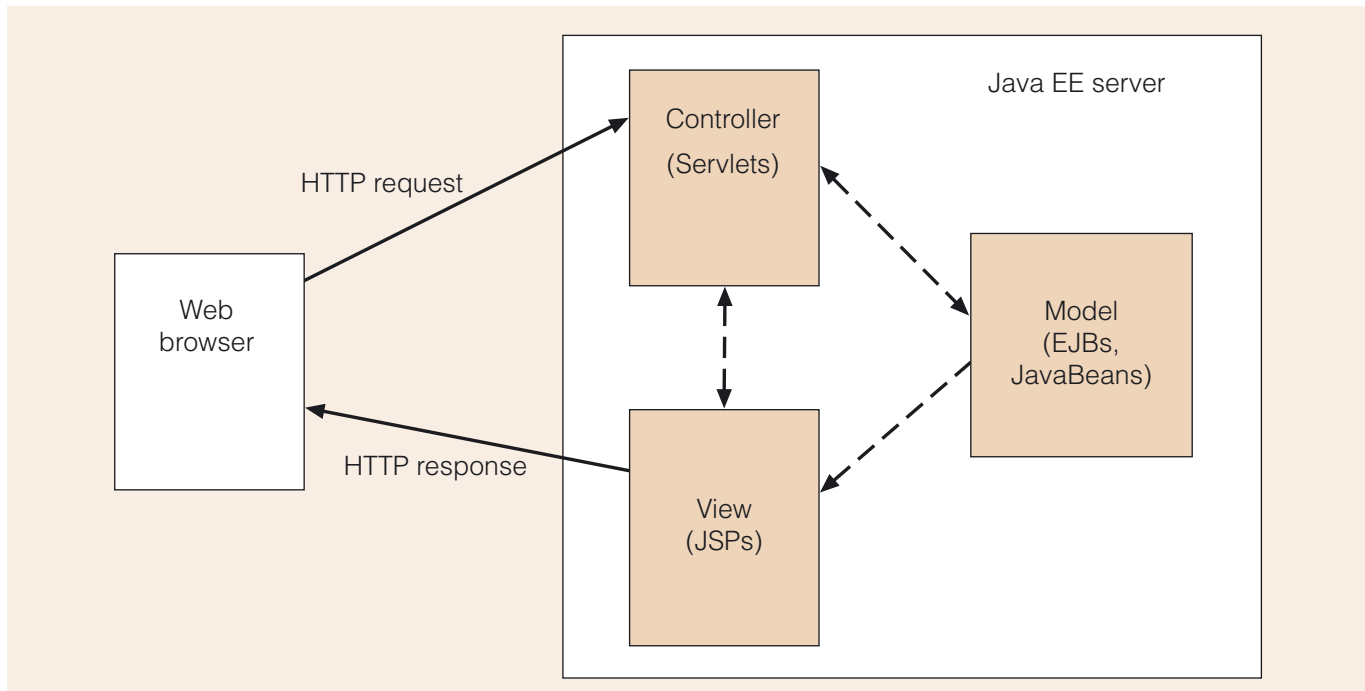


Figure 14 A simple MVC approach to web applications (dashed arrows show data flow within the server)

6.2 Communication between components in an MVC architecture

The division of labour outlined by the MVC approach seems reasonable, but so far we have not described in any detail how the three categories of component interact. There are two aspects to this interaction: control and communication. We outline first the responsibilities of each category.

- ▶ The **model** represents business data and associated business logic which allows access or modification of the business data. It should be possible for the view to query the model to find out its current state. For example, if the model includes an ecommerce shopping cart, the view can show the user the current state of their shopping cart. It may be useful for the model to notify the view when any of its data (i.e. the state of the model) changes.
- ▶ The **view** makes the contents of the model visible in an appropriate way – in web applications this normally means constructing one or more dynamic web pages. The view must be able to query the data in the model and must update the presentation when the model changes. There are some important issues about how often the view changes in response to changes in the model – some of these issues are explored in more detail in the next unit.
- ▶ The **controller** regulates the overall behaviour of the application. It may receive user inputs directly or the inputs may have been forwarded by view components – in a web application the user inputs are mainly HTTP `GET` or `POST` requests. As a result of these requests, it may interact with the model, potentially invoking actions by the model such as updating some data. It also decides which view components are displayed in response to requests and any resulting internal operations.

Control and communication in servlets

Control mainly involves servlets (or JSP pages) invoking other servlets (or JSP pages) to carry out part of the work involved in responding to a request. Communication relates to the various MVC components sending or receiving data as shown in Figure 14.

First we look at how this control is carried out in servlets, using a **request dispatcher**. This is an object of type `RequestDispatcher` that can be obtained from the servlet request object, for example as follows.

```
protected void doGet(HttpServletRequest req,
                    HttpServletResponse resp)
    throws ServletException, IOException
{
    ...
    RequestDispatcher disp =
        req.getRequestDispatcher("displayResults.jsp");
    if (disp != null)
    {
        disp.forward(req, resp);
    }
    ...
}
```

By invoking the `forward` method of the `RequestDispatcher` object, we pass the request to the JSP component `displayResults.jsp` so that the JSP page can construct the response. When forwarding is used in this way, the servlet is delegating construction of the HTTP response completely to the destination component (which can be either a JSP page or another servlet).

Hence the `forward` method must be called before any output is sent to the response stream. Also, the servlet must not attempt to add later to the response in the servlet after the invocation of the `forward` method.

An alternative approach is to use the `include` method of the `RequestDispatcher` object, to delegate *some* of the response construction to another component. Using the above example, the invocation of the `forward` command would be replaced as follows.

```
disp.include(req, resp);
```

This causes any response created by the included component to be included in the response returned by the servlet. Therefore it is perfectly permissible for the servlet to send items to the response before and after the invocation of the `include` method.

It is often useful to pass some information to an included or forwarded component. This can be done by adding items to the request object, in a similar way to adding items to a session object, as we saw earlier. To add an item referenced by `shoppingCart` to the request object `req` we proceed as follows.

```
req.setAttribute("cart", shoppingCart);
```

To retrieve this we use the `getAttribute` method as follows.

```
Cart shoppingCart = (Cart)req.getAttribute("cart");
```

Note the need to cast the retrieved attribute to the correct type (in this case `Cart`). This is because the request object stores the attributes as references of type `Object`.

Control and communication in JSP

JSP also provides ways of invoking other servlets (or JSP pages) to carry out part of the work involved in responding to a request. Since JSP pages can contain scriptlets with arbitrary Java code, we could of course just program this in Java as we did for servlets in the previous subsection. However, this is not really in the spirit of keeping JSP pages as free of Java coding as possible.

Fortunately JSP provides a number of action tags which provide exactly what we need – the standard actions `<jsp:forward>` and `<jsp:include>`.

With the `<jsp:include>` action, the JSP page passes the HTTP request to the included file. The included file passes its output to the response. Then control returns to the calling JSP page, which finishes processing the response. Components that are included via the `<jsp:include>` action must be valid JSP pages or servlets. This is exactly the same behaviour as we saw above for servlets using a `RequestDispatcher` object.

Here is an example of the use of this action, in the form of a single tag:

```
<jsp:include page = "displayResults.jsp" />
```

Data can be passed to the included file if needed by means of parameters, for example:

```
<jsp:param name="NumberOfUsers" value="<%= userCount %>" />
```

Here the parameter name is a string associated with a value which can be a simple fixed string or the result of a Java expression – in this case, the value of integer variable `userCount`.

To pass data in this way, the `<jsp:include>` action is split into start and finish tags to enclose any parameter tags, for example:

```
<jsp:include page = "displayResults.jsp" >
  <jsp:param name="NumberOfUsers" value="<%= userCount %>" />
  <jsp:param name="DBTable" value="Stock_table" />
</jsp:include>
```

The `<jsp:forward>` action is similar to the corresponding `forward` method of the `RequestDispatcher` object for servlets. It passes the HTTP request to the specified JSP page or servlet for the destination component to completely construct the response. As with its servlet equivalent, if the forwarding occurs, the calling JSP page must not output anything at all to the response. To be specific, this means that there must be no text, HTML or standard XML (as opposed to JSP tags) occurring in the JSP file before the `<jsp:forward>` action. Also there must be no output to the response as a result of JSP scriptlets or expressions. The same restrictions apply to the JSP file after the forwarding component has completed its output to the response stream.

The syntax for the `<jsp:forward>` action is similar to that for the `<jsp:include>` action, for example:

```
<jsp:forward page = "displayResults.jsp" >
  <jsp:param name="NumberOfUsers" value="<%= userCount %>" />
  <jsp:param name="DBTable" value="Stock_table" />
</jsp:forward>
```

Note that the `<jsp:include>` action is completely different from the `include` directive that we saw in an earlier section for inserting blocks of standard JSP text from another file into the current JSP page. Recall that its format was as follows.

```
<%@ include file = "std_header.jsp" %>
```

Or, using the XML-style syntax for JSP directives:

```
<jsp:directive.include file = "std_header.jsp" />
```

JSP files that are included in this way do not have to be a complete JSP page themselves – they are normally fragments of JSP such as standard page headers or footers.

Other means of communication – using scope objects

We have seen that servlets or JSP pages can use parameters to pass data to another component which they are including or forwarding to. Another option, which is very commonly used, is to associate data with a scope object as an attribute. We have already met a number of examples where data was stored in the scope object associated with the request or the session and later retrieved during a subsequent request, or by another component of the application. The four standard scope objects for servlets and JSP are summarised in Table 5 together with their accessibility.

Table 5 Scope objects for servlets and JSP – based on Sun Microsystems (2007a)

Scope object	Class (all from <code>javax.servlet</code>)	Accessible from
Web application	<code>ServletContext</code>	All web components within a web application.
Session	<code>http.HttpSession</code>	Web components handling a request that belongs to the session.
Request	<code>HttpServletRequest</code>	Web components handling the request.
Page	<code>jsp.JspContext</code>	The JSP page that creates the object.

JSP has a number of implicit objects that allow access to these scope objects – these were listed in Table 3 in Subsection 5.2, and we saw some examples of how to access their attributes using JSP EL via the implicit objects in Table 4.

In JSP pages that do not use EL, you can use Java scriptlets that access the scope objects in the same way that servlets do. The page scope object applies to JSP only and not to servlets.

In servlet programming the various scope objects are normally available as global objects (the web application context) or from the request object passed to the servlet. For example, if we have a request object of type `HttpRequest` passed to the servlet as a parameter named `request`, the following code will access the session object.

```
HttpSession session = request.getSession();
```

The web application context can be accessed either directly or via the session object as follows.

```
ServletContext context1 = getServletContext();
ServletContext context2 = session.getServletContext();
```

Object attributes can be stored in or retrieved from each of these objects using the `setAttribute` and `getAttribute` methods. The signature of each of these methods is exactly the same for all four standard scope objects.

Which scope object is appropriate for storing and passing data depends on which web components (servlets or JSP pages) will need access to it and how long the data must be maintained. Broadly speaking, Table 5 is arranged with the scope objects in descending order of lifetime and accessibility.

- The web application context object is the most general and widely accessible object, being available to all components in the application irrespective of which client session or which request they are associated with.

- ▶ The session object is available only to components associated with servicing requests from a particular client session, but data will need to be maintained over as many requests as that session requires.
- ▶ The request object stores data associated only with a particular request – this data will be lost once the response to that request has been sent.
- ▶ The page scope is the narrowest – the page scope object can hold data associated with a particular JSP page only. It is not relevant for servlets.

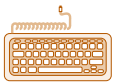
As most of these scope objects are shared between many components, there is a need to consider carefully where any shared data stored in such objects may need to be protected to ensure safe concurrent access.

SAQ 6

- (a) What are the three main components of the MVC approach?
- (b) How can this approach be applied in web-based information systems using Java EE?

ANSWER.....

- (a) They are the model, the view and the controller. The model represents business data and associated business logic. The view makes the contents of the model visible. The controller regulates the overall behaviour of the application.
- (b) In a Java EE application the model typically uses EJBs and JavaBeans. In web applications the view component normally refers to constructing one or more dynamic web pages using JSP. The controller may receive user inputs directly or forwarded via view components. In a web application the user inputs are mainly HTTP GET or POST requests – typically the controller will consist of one or more servlets. As a result of these requests, it may interact with the model, potentially invoking actions by the model such as updating some data. It also decides which view components are displayed in response to requests and any resulting internal operations.



Activity 8.7

MVC for the web tier.

6.3 MVC frameworks

There are a number of **application frameworks** such as Apache Struts and JavaServer Faces that are intended to help with the construction of web applications following the MVC approach. Typically, these include a configurable 'front controller' servlet, and provide abstract classes that can be extended to handle request dispatches. Some frameworks include macro languages or other tools that simplify application construction.

Apache **Struts** is a free open-source framework for creating Java web applications. The framework provides three key components.

- ▶ An extensible 'front controller' that dispatches requests to an 'action' handler provided by the application developer.
- ▶ A location handler that transfers control to another resource that completes the response.
- ▶ A tag library that helps developers create interactive form-based applications.

Struts has been used for a large number of open-source and commercial web applications.



Apache Struts framework.

JavaServer Faces (JSF) builds on the experience of the Struts project as well as that of most of the technologies we have seen so far in this unit – JSP, servlets, tag libraries and so on. It emphasises ease of use and clear separation of functions (as in MVC), while making it easy to link the various components of the application. It provides the following.



JavaServer Faces.

- ▶ A set of APIs for representing user interface components and managing their state, handling events and input validation, defining page navigation, and supporting internationalisation and accessibility.
- ▶ A JSP custom tag library for expressing a JSF interface within a JSP page.

According to Sun Microsystems (2007b):

Ease-of-use being the primary goal, the JavaServer Faces architecture clearly defines a separation between application logic and presentation while making it easy to connect the presentation layer to the application code. This design enables each member of a web application development team to focus on his or her piece of the development process, and it also provides a simple programming model to link the pieces together. For example, web page developers with no programming expertise can use JSF user interface component tags to link to application code from within a web page without writing any scripts.

It is important to be aware of these frameworks for serious large-scale development. For the purposes of this course, we will not consider them any further, but you can read more about them from the links provided on the course website if you wish.

SAQ 7

- (a) What is the primary role of servlets in a web-based information system?
- (b) When should you use JSP?
- (c) How can servlets and JSP be combined in complex web-based systems?

ANSWER.....

- (a) Servlets are useful for more complex processing, where the static content of the response is very limited or is delegated to another component.
 - (b) JSP is particularly suited to constructing dynamic web pages in situations where the output is predominantly static HTML or XML.
 - (c) Both technologies are used by a server to create a web page in response to a client request. Servlets and JSP are complementary in their strengths and weaknesses, and most systems of significant complexity would require a combination of both. Servlets can invoke JSP pages and vice versa so that both can carry out the aspects of the response to which they are best suited. The MVC pattern is usually recommended as a guideline for the roles of servlets and JSP in web applications.
-

7

Communication with the business tier

The web tier sits between the client tier and the business tier. Communication with the client tier is by HTTP requests and responses – we will see more of how this works from the client point of view in the next unit. We have also seen that a simple approach to business logic is possible using JavaBeans or using JDBC directly. The web tier communicates with JavaBeans objects by normal Java programming in servlets, or by using the JSP actions, such as `<jsp:usebean>` outlined in Section 4. Similarly, access to data via JDBC can be programmed directly in Java servlets, or using the `<sql:>` tags from the JSTL.

While this may suffice for simple systems, larger-scale systems require clearer separation of functions and we have noted earlier that `<sql:>` tags have been deprecated. In Java EE this separation normally means using EJB session beans to handle business logic. Recall that, although they have some things in common, including the 'bean' part of their name, EJBs and JavaBeans are quite different entities. EJBs run in a special EJB container that handles many important aspects of distributed enterprise systems – such as transactions, authorisation, authentication, and so on.

Following the MVC approach outlined in the previous section, the main communication from the web tier to the business tier should be carried out by the controller, which normally consists of one or more servlets. Recall that EJB session beans have two main components:

- ▶ a business interface that defines the methods offered by the session bean;
- ▶ a bean class that implements the methods defined in the interface.

The bean class can potentially reside on a remote machine, but this makes little difference to the coding of the components that use the session bean. See *Unit 7*, Subsection 5.2, for a discussion of the issues around local access and remote access to EJBs. Figure 15 shows how a controller servlet interacts with a session bean via the bean's business interface.

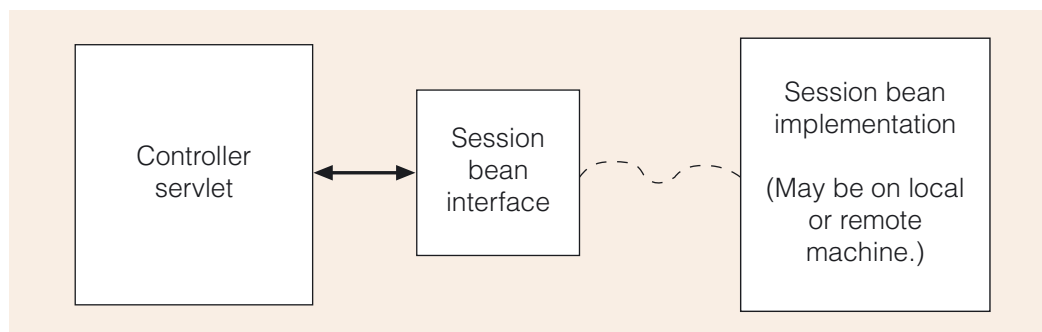


Figure 15 Interaction between servlet in the web tier and session bean in the business tier

As an example of interaction between the web tier and session beans let us consider a session bean that represents a shopping cart in The Music Store ecommerce application. Assuming that we are using local access to this session bean, we can call its local interface `CartLocal`. We also need an implementation class, which we will call `CartBean`. This should be a stateful session bean to allow it to conveniently store the details of the shopping cart in the system database. We omit some of the details as these have been explained in the previous unit on the business tier. A suitable local interface is as follows.

```
import javax.ejb.Local;
@Local
public interface CartLocal
{
    boolean addToCart(Instrument instr);
    boolean removeFromCart(Instrument instr);
    boolean checkout();
    java.util.List<Instrument> getCart();
}
```

This interface is to be implemented by the class `CartBean`, so this class must contain the method code for all of the methods listed in the interface `CartLocal` – we omit the details here as they have been discussed in *Unit 7*. We note only that operations on the shopping cart do not affect the underlying database until the `checkout` method is invoked – until then the shopping cart is simply an object stored in memory, and the user can add or remove items from the cart without any permanent effect.

If a servlet wishes to access this session bean, for example to add a new item to the shopping cart as a result of a user request, it must carry out a number of operations as follows.

- ▶ Access the shopping cart bean – it can do this by dependency injection using the `@EJB` annotation.
- ▶ Identify the item to be added – we assume that a unique instrument ID is passed to the servlet as a request parameter.
- ▶ Find the details of the instrument in the stock database.
- ▶ Add the instrument object to the shopping cart.
- ▶ Mark the instrument as reserved (but not sold until the user confirms by checking out).

Dependency injection was explained in *Unit 7*.

The servlet code for this must reside in the `doPost` method, since it is making a change to the shopping cart data. An example of the code required is as follows.

```
public class AddToCart extends HttpServlet
{
    // This gives access to methods of the instrument façade bean.
    @EJB
    private InstrumentFacadeLocal instrumentFacade;

    // This gives access to methods of the cart bean.
    @EJB
    private CartLocal shoppingCart;

    protected void doPost(HttpServletRequest req,
                           HttpServletResponse resp)
        throws ServletException, IOException
    {
        // Store the cart bean in the current session object
        // so that other servlets can use the same cart.
        HttpSession currentSession = req.getSession();
        currentSession.setAttribute("cart", shoppingCart);

        // Get instrument ID from the request
        // that invoked this servlet.
        String instrId = req.getParameter("ID");
```

```

        // Locate the Instrument object corresponding to this ID.
        Instrument instr =
            instrumentFacade.find(new Integer(instrId));

        boolean wasAdded = shoppingCart.addToCart(instr);
        if (wasAdded)
        {
            instr.setSold("reserved");// Update instrument status
        }
    }
}

```

There are a number of points to note in this example.

The shopping cart EJB component referenced by `shoppingCart` is made an **attribute** of the session object. This ensures that it can be accessed by other servlets or JSP pages involved in responding to client requests during the current session.

We are also making use of an EJB called `InstrumentFacadeBean` that has a local interface called `InstrumentFacadeLocal`. This gives indirect access to the details of instruments stored in the underlying database for the application. We need not be concerned with all the methods of this interface here, but we do invoke the method declared as follows.

```
Instrument find(Object key);
```

This method allows us to search the database for the instrument details corresponding to the given primary key value (in this case the instrument ID). Although the underlying database is a relational database, and is not based on objects, this returns an object of type `Instrument` corresponding to this ID (if there is one). Here we are making use of the persistence approach outlined in *Unit 7* which maps relational database tables to objects in an appropriate way.

In this example, the shopping cart is stored in memory as a session attribute. When the `checkout` method of the `CartLocal` interface is invoked, this will update the database to set the status of all instruments in the cart as sold. This would happen in some other servlet as a result of a user request.

By contrast, operations on instruments do affect the underlying database, because of the persistence features built into the `InstrumentFacadeBean` and the `Instrument` class.

The EJB annotations used for dependency injection are a convenient way of causing the web container to make the links between the servlet and the specified EJBs. Prior to Java EE version 5 these facilities were not available and it was necessary for servlets to use the Java Naming and Directory Interface (JNDI) to access (or 'lookup') resources such as EJBs. We will see in *Unit 9* that this JNDI lookup may still be necessary for application clients if they are not running in the environment of an application client container.

JNDI was explained in *Unit 7*.

8

The Music Store scenario

The current scenario for The Music Store application is as follows.

The Music Store

A music shop sells second-hand and rare musical instruments. The instruments are stored in a warehouse from where they can be shipped to customers who have purchased them. There is a server computer in the music shop so staff can keep the catalogue updated. There is also a computer in the warehouse to help keep track of goods in and out and stock currently in the warehouse.

The server in the shop is accessible to the public via the internet from anywhere. The shop also has to deal with payments – the shop server has to communicate with various financial systems in order to deal with payments.

Customers register with the system before use, entering contact details and financial details so that they can pay for purchases. At present we ignore the details of how payments are handled, assuming that this is outsourced to another organisation.

The shop system has software to allow customers to buy items over the internet. Each item has a fixed price and no bargaining or bidding is possible.

A customer selects one or more items that they wish to purchase. These items are added to an electronic shopping cart. When they have finished selecting items, they can move to the checkout page where they are informed of the cost, and they arrange to pay.

Once the system has accepted payment, it authorises the warehouse to despatch the instrument(s).

Figure 16 shows an example of how the shopping cart might look in this system.



Activity 8.8
Applying web tier ideas
to The Music Store
scenario.

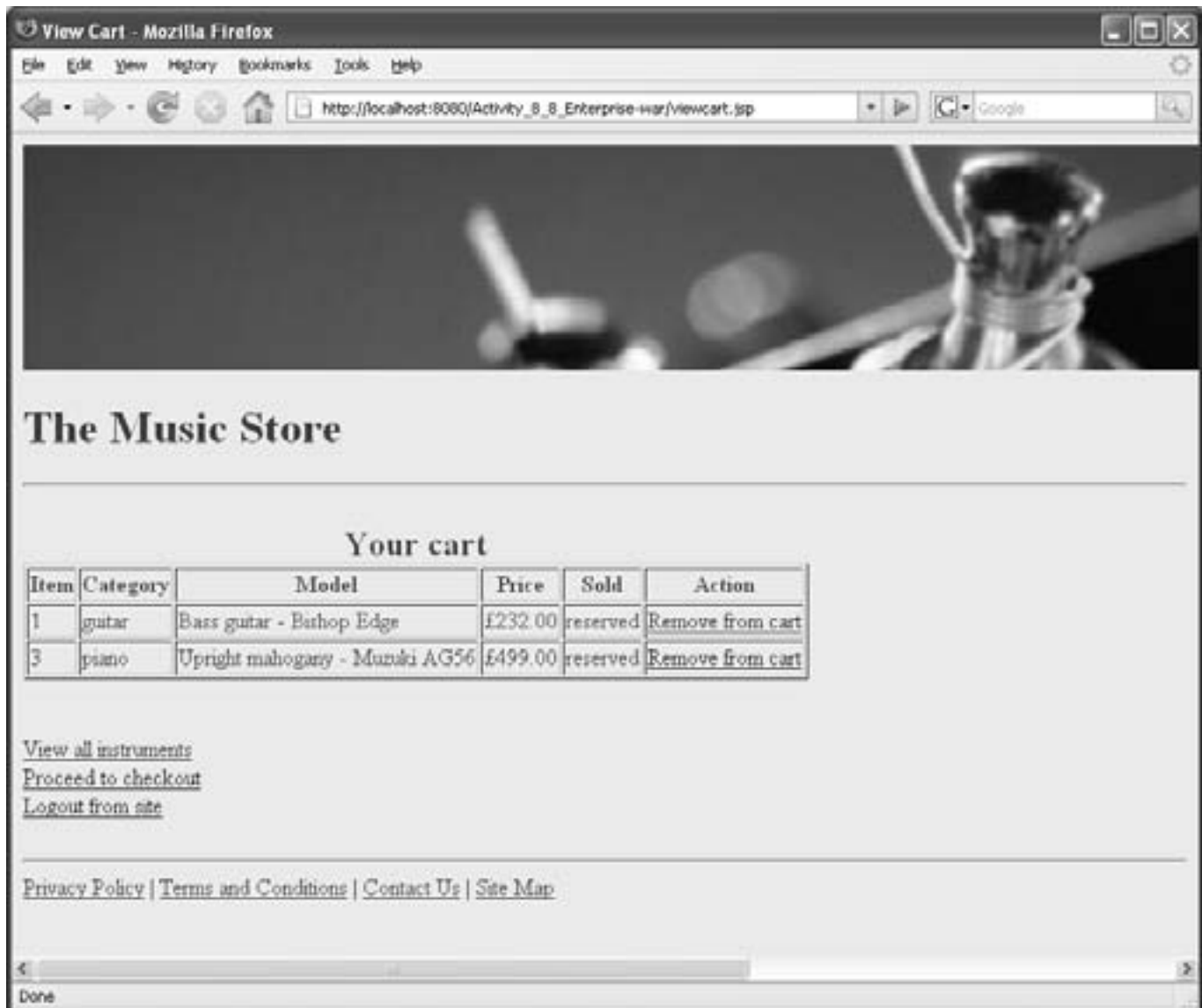


Figure 16 Example view of the shopping cart for The Music Store scenario

9

Summary

In this unit we have explained the role of the web tier in a Java EE application in providing a dynamic web interface. We also considered the main components of the web tier, Java servlets and JSP pages, and how they work. The web tier communicates with the client tier by means of HTTP requests and responses. It normally communicates with the business tier by invoking operations on EJB session beans.

Servlets are Java programs that run on the server – this has advantages in terms of security, flexibility and scalability over previous technologies such as Java applets or other client-side processing. However, servlet programmers must be aware of issues around concurrency, and shared data and resources. It is also necessary to cater for the stateless nature of the HTTP protocol by managing sessions, using cookies or other means to help preserve state information relevant to a series of client requests.

JSP pages are a mixture of static items such as text, HTML or XML, together with dynamic items, that allow web pages to be constructed dynamically as the application runs. In classic JSP the dynamic items mainly take the form of fragments of Java code known as scriptlets; more recently JSP authors have been encouraged to use alternative predefined libraries of JSP tags such as the JSTL and the JSP EL. Use of these libraries helps to keep presentation (JSP tags and EL) separate from the processing (Java programming) – such separation aids maintenance and allows development staff to specialise in just one aspect of web tier technology. Since JSP pages are translated to servlet-like code for execution, many of the same issues about concurrency and shared data apply equally to JSP pages as to servlets.

Simple applications may have presentation and processing combined in a small number of servlets or JSP pages – this is known as the Model 1 approach. The recommended approach for sizeable systems is the MVC pattern, also known as the Model 2 approach. This provides a clear separation of presentation and processing, often using a controller servlet to coordinate the other components. There are standard MVC frameworks, such as Apache Struts or JavaServer Faces, which are intended to facilitate development of web applications using this MVC approach. Both servlets and JSP pages have facilities to invoke other servlets or JSP pages to carry out some of the processing or presentation in more complex systems. There are also a variety of means of sharing data between servlets and JSP pages, and across client sessions – careful attention must be paid to synchronisation of any updates to this shared data, where appropriate.

In the next unit we look at the client tier.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ explain the main functions of the web tier;
- ▶ describe the need for servlets and explain how they work;
- ▶ program simple servlets to respond to HTTP requests;
- ▶ explain the role of JavaBeans in Java EE applications;
- ▶ understand how JSP technology can be used to generate web pages dynamically;
- ▶ write simple JSP pages to display dynamic web pages;
- ▶ distinguish when to use servlets or JSP, or a combination of both;
- ▶ implement a simple MVC approach for a web application;
- ▶ outline the means of communication between the web tier and adjacent tiers;
- ▶ discuss concurrency and distribution issues for the web tier.

Glossary

applet A block of compiled Java (in bytecode form) that can be downloaded along with a web page and that runs within suitable browsers.

application framework A set of classes, libraries and other software that provides a structure to help construct systems for standard types of application – for example, Apache Struts or JavaServer Faces are web application frameworks.

attribute (of a scope object) A data item associated with a scope object in a servlet or JSP page.

business tier In Java EE systems, the part of the software running on the server that deals with business logic and provides the interface to the enterprise information system (database) tier.

Common Gateway Interface (CGI) A standard for interfacing external applications with information servers, such as HTTP or web servers. CGI programs are stored on the web server and are run by that server in response to particular HTTP requests.

cookie An item of text that a web server may send to a client web browser as part of its response to an HTTP request. The web browser stores the cookie text, usually in a small disk file. This has many uses, such as making it easier for users to log in to a site they have visited previously or helping to maintain session information.

custom action A user-defined JSP tag in XML format that specifies some sort of action to be carried out by the JSP page – it actually invokes some Java code from a custom tag library.

custom tag See **custom action**.

deployment descriptor An XML file (automatically generated by IDEs) that defines the internal structure of each Java EE module and allows the setting of a number of attributes of the module, such as transaction behaviour, security settings, etc.

directive tag A JSP element such as `page`, `include` or `taglib` that provides information to the JSP container about the page, and affects how the JSP compiler generates the servlet that implements the JSP page.

fat client Software that runs on the client machine of a client-server system to provide a considerable degree of functionality, such as a complex user interface and significant client-side processing.

idempotent operation An operation that can be safely repeated because execution of the operation should leave the system in the same state – for example, a database query.

implicit object An object in a component such as a JSP page or a servlet that is already available for access and does not need to be explicitly declared, e.g. a request object.

JavaBean An object of a Java class with the following characteristics: a zero-argument constructor, getter methods for every readable property, setter methods for every writable property, must be serialisable. JavaBeans are useful in web applications for storing, or transferring, data between servlets (and JSP pages).

JavaServer Faces An MVC application framework for use in constructing web applications.

JavaServer Pages (JSP) A technology that facilitates creation of web pages with **JavaServer Pages pages**.

JavaServer Pages page (JSP page) A web page with both static and dynamic components that can be used to dynamically generate a web page in response to an HTTP request. These JSP pages are complementary to servlets and may be more convenient for web designers as they can be created without Java programming. See also **JavaServer Pages (JSP)**.

JSP document A JSP page which uses only XML-style tags, hence allowing the JSP page to be manipulated and validated by XML tools.

JSP expression A JSP element containing a Java expression that produces a value; this value will be displayed as part of the output (different from the **JSP expression language** since a JSP expression uses Java syntax directly).

JSP expression language A language for use in JSP to write expressions that access JavaBeans components and a number of implicit objects – an alternative to using Java programming fragments.

JSP Standard Tag Library (JSTL) A JSP tag library forming a standard part of the Java EE that contains a number of commonly used sub-libraries (such as the core library, for flow control, and libraries containing tags facilitating XML processing, internationalisation and SQL database access).

JSP tag An element in a JSP page, enclosed in angle brackets, that is part of specifying the dynamic content of the page – the three main categories of JSP tag are directive, scripting and action.

JSP tag library A collection of Java code that carries out the actions associated with JSP tags, especially custom tags, hence relieving JSP developers of much of the need for Java programming in the JSP pages. The most commonly used tag library is the JSTL.

lifecycle method One of several methods that are defined for all servlets to deal with initialisation, handling requests or final clean-up.

Model 1 An approach to web application design in which components may include a mixture of control logic, presentation logic, and sometimes business logic.

Model 2 An approach to web application design which uses MVC to separate each of the control logic, presentation logic and business logic into different components.

model-view-controller (MVC) A design pattern that structures a system using three categories of component: model, view and controller. The model is the abstraction of all the domain data in the system. The view is the visualisation of the model. The controller is the set of components that manage the communications between model and view.

multithreaded servlet A servlet that is expected to handle concurrent requests by running multiple threads if necessary – this is the default behaviour for servlets.

parameter (in an HTTP request) A named item of data that is sent as part of the HTTP request as a name-value pair and can be extracted for processing by a servlet or JSP page.

property (of a JavaBean) This loosely corresponds to the instance variables or attributes of a JavaBean class. Usually the property is implemented directly as an instance variable but it need not be. JavaBeans must have a getter and setter method for each property.

protocol (for communication) The rules used in a communication, such as HTTP for the Web.

request dispatcher An object available to servlets (and JSP pages) that can be used to send an HTTP request to another servlet (or JSP) to produce some, or all, of the response.

request-response A communication approach between software components where one sends a request and the other returns a response, for example HTTP.

safe method A method without any side effects for which users are held responsible – for example, a database query because it does not change any stored data.

scope (of an object) The extent over which an object or other item is valid or accessible. For example, items that have session scope continue to exist throughout the duration of the session in which they were created.

scope object An object associated with a particular scope in a web application, such as the JSP page, the request, the session, or the whole application. Scope objects can be used to store or transfer data between components such as JSP pages and servlets. See also **attribute**.

scripting tag An element in JSP, with several possible types, that allows inclusion of Java declarations, Java executable statements, or Java expressions in the JSP page.

scriptless JSP pages can be specified as scriptless so that no Java-style scripting tags will be accepted by the compiler, hence enforcing use of JSP EL and JSP tags.

scriptlet A fragment of Java code in a JSP page, consisting of one or more complete Java statements that will be executed when the JSP page runs.

servlet A Java program that runs on the server and that follows a request-response model, normally responding to HTTP requests.

session A conversation consisting of a series of linked requests and responses between the client and server on behalf of a particular user.

session ID A number or string that uniquely identifies a particular session and any associated data.

single-threaded servlet A servlet that is limited to handling only one request at a time – this approach is not now recommended.

standard action One of the JSP tags that form a standard part of the JSP specification and that cause some sort of action to be carried out by the JSP page (as opposed to custom actions which are user-defined via custom tag libraries).

stateless protocol A communication protocol, such as HTTP, where no state information is stored and each request or other communication is independent of any others.

Struts framework An MVC application framework for use in constructing web applications.

tag An element of a markup language, such as HTML or XML, or in JSP, with a variety of meanings and uses. In JSP, tags are mainly used to include fragments of Java code, or equivalent items such as JSTL and JSP EL, in order to generate the dynamic parts of web pages.

tag handler The Java code that is executed when a JSP page containing its associated custom tag is run – this code is stored in a tag library.

thread-safe Shared data, or program components that access it, are thread-safe if there is no possibility of data loss or corruption when a number of concurrent threads are active.

trusted applet An applet that can guarantee its origination from a known source by means of cryptography, and may therefore be permitted to carry out a wider range of actions than an untrusted applet.

untrusted applet The default situation for an applet automatically downloaded along with a web page and subject to severe restrictions in the interests of security.

web container A software environment in which servlets and JSP pages run, and that provides a number of services including security, concurrency and management of the servlet lifecycle.

web form A portion of a web page that allows the user to enter data to be sent, in the form of an HTTP request with associated parameters, for processing by the server.

web tier The part of the middle tier in a Java EE application that communicates with the client tier, typically by receiving and responding to HTTP requests using dynamically generated web pages.

References

- Burbeck, S. (1992) *Applications Programming in Smalltalk-80TM: How to use Model–View–Controller (MVC)* [online],
<http://st-www.cs.uiuc.edu/users/smarch/st-docs/mvc.html> (Accessed 15 July 2007).
- Korpela, J. (2003) *Methods GET and POST in HTML Forms – What’s the Difference?* [online], <http://www.cs.tut.fi/~jkorpela/forms/methods.html> (Accessed 15 July 2007).
- Sun Microsystems (2007a) *The Java EE 5 Tutorial* [online],
<http://java.sun.com/javaee/5/docs/tutorial/doc/bnafo.html#bnafp>
(Accessed 3 October 2007).
- Sun Microsystems (2007b) *JavaServer Faces Technology Overview* [online],
<http://java.sun.com/javaee/jaserverfaces/overview.html>
(Accessed 3 October 2007).

Index

A

- applet 7
 - trusted 7
 - untrusted 7

- application framework 48
 - JavaServer Faces 49
 - Struts 48

- attribute (of a session object) 20, 28, 52

B

- business tier 6

C

- Common Gateway Interface (CGI) 9

- cookie 20

F

- fat client 6

H

- HTTP 6, 9, 12

- HttpServlet 12

I

- idempotent operation 16

- implicit object 27, 38

J

- JavaBean 31, 35
 - property 31, 39, 41

- JavaServer Pages (JSP) 6
 - custom action 24
 - custom tag 36
 - deployment descriptor 41
 - directive tag 24
 - document 25
 - expression 24
 - expression language (EL) 36, 42
 - scripting tag 24
 - scriptless 41
 - scriptlet 24
 - standard action 24
 - Standard Tag Library (JSTL) 36
 - tag 24
 - tag handler 36
 - tag library 36

M

- Model 1 and Model 2 approach (for web tier) 43

- model-view-controller (MVC) pattern 43

P

- parameter (in an HTTP request) 14, 33–34, 39

- protocol (for communication) 8
 - stateless 9

R

- request-response 8

S

- safe method 16

- scope (of an object) 32, 39

- scope object 39, 47

- servlet 6

- lifecycle method 11
 - multithreaded 10
 - request dispatcher 45
 - single-threaded 10

- session 9, 19–20

- session ID 20, 40

- session object 28

T

- tag 23

- thread-safe 15, 17–18

W

- web

- container 10
 - form 7–8

- web tier 6, 10, 41

- Model 1 approach 43
 - Model 2 approach 43
 - model-view-controller (MVC) pattern 43